ADVANTEST

ADVANTEST CORPORATION

SmartShell System Reference

Command Reference

SmarTest 7.0

2023-08-28

Notices

Legal Notices

Trademarks and Registered Trademarks

Warranty

Technology Licenses

Restricted Rights Legend

Safety Notices

⚠ CAUTION

A CAUTION notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in damage to the product or loss of important data. Do not proceed beyond a CAUTION notice until the indicated conditions are fully understood and met.

⚠ WARNING

A WARNING notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in death or severe injury. Do not proceed beyond a WARNING notice until the indicated conditions are fully understood and met.

Table of Contents

List of Figures

1 Start a SmarTest session or Connect to a running SmarTest session

The command `INIT` can either start a new SmarTest session with the specified settings and executes the specified test program and connect to an existing running SmarTest session. Depending on the complexity of the project, it might take some time.

```
INIT SESSION=<session-id> TESTPROGRAM=<test-program-name>
FLOW=<testflow-name> MODEL=<model-file-name> MODE=<online/offline>
```

2 Test Program Development

SmartShell commands are grouped into command groups for pattern generation and modification, test program generation and execution, testflow variable and test specification variable modification, hardware setting modification and other miscellaneous command sets.

2.1 Pattern Generation

2.1.1 Basic Pattern Generation

```
SET PAT <label_name> CONSTR<port_name>
SET SIGNAL_LIST <signal list>
SET VEC         R<n> <data>
SET VEC_END     Temporary end
SET PAT_END     <label_name>[+,]
SET OPSEQ       <spb_label> SEQ <> PORT=<> LABEL_LIST=<>[+,]
SET PSEQ        <mpb_label> PAR LABEL_LIST=<spb_label>[+,]
```

With "`SET VEC_END`", the transferred vector data is held in cache and can be resumed later. With "`SET PAT <> CONSTR <>`" command, the data transfer of the same pattern continues.

With "`SET PAT_END`", the vector data transfer is completed and it is ready for conversion.

With "`SET OPSEQ SEQ`", a list of one or more patterns under the same signal group is generated and with "`SET OPSEQ PAR`", multiport burst label is generated.

Example:

```
SET PAT scan_pat1 CONSTR jtag_port
SET SIGNAL_LIST pin1,pin2,pin3,pin4
SET VEC R1  0000
SET VEC R10 1010
SET VEC R1  1010
SET PAT scan_pat2 CONSTR jtag_port
SET SIGNAL_LIST pin1,pin2,pin3,pin4
SET VEC R3  0000
SET VEC R1  1010
SET PAT ctrl_pat CONSTR ctrl_port
SET SIGNAL_LIST pin5,pin6,pin7,pin8
SET VEC R1  0000
SET OPSEQ seq1 SEQ grp1 LABEL_LIST=scan_pat1 scan_pat2
```

```
SET OPSEQ seq2 SEQ grp1 LABEL_LIST=ctrl_pat
SET OPSEQ scan_test PAR LABEL_LIST=seq1 seq2
```

2.1.2 State Character Mapping Selection

It is quite often that the existing timing wavetable in V93000 has multiple state maps. Currently, the interface supports the following mapping:

```
PINS CLOCK
0 "{ d1:1 d2:F0N }" P
1 "{ d1:1 }" 1
2 "{ d1:0 }" 0
3 "{ d1:Z }" Z
4 "{ r1:X }" X
brk ""

STATEMAP
#physSeq statechar period selector
0 P x1 project plltst_d_jtag_p_12/P TS1/P
1 1 x1 project plltst_d_jtag_p_12/1 TS1/1
2 0 x1 project plltst_d_jtag_p_12/0 TS1/0
3 Z x1 project plltst_d_jtag_p_12/Z TS1/Z
4 X x1 project plltst_d_jtag_p_12/X TS1/X
```

Fig. 1: STATEMPA Selector Type A

To use plltst_d_jtag_p_12 mapping table, the command is

```
SET STATEMAP    plltst_d_jtag_p_12
```

The sample script looks like this:

```
SET PAT CONTR scan_pat
SET STATEMAP    plltst_d_jtag_p_12
SET SIGNAL_LIST pin1,pin2,pin3,pin4
SET VEC R1  0000
SET VEC R10 1010
SET VEC R1  1010
SET VEC R1  1010
SET VEC R1  1010
```

SmartShell interface will be enhanced to support another STATEMAP format:

```
PINS pin1
0 "{d1:0}"    0ts1  # 0
1 "{d1:1}"    1ts1  # 1
20 "{d3:0}"    0ts2  # 0
30 "{d3:1 d4:F0N}"    1ts2  # 1
brk ""

STATEMAP
#physSeq    statechars    x-mode    selector
0           0             x1        tset2
1           1             x1        tset2
20          0             x1        tset1
30          1             x1        tset1
```

Fig. 2: STATEMPA Selector Type B

Each pin or group may have different mapping. This format will be supported in future versions.

2.1.3 Speed-up Vector Data Transfer

The `SET TOGGLE` commands are used to reduce the redundancy data to be transferred repetitively when only a few data differ from the previous vector data to speed up the data transfer.

```
SET TOGGLE R1 1@0 5@H
```

The `SET NOP` command tells the SmartShell server to keep adding the same vector to the pattern set by the `SET VEC` command immediately before.

```
SET NOP
```

2.2 Pattern Modification

2.2.1 Edit Vector Data

EDIT VEC has multiple capacities:

- Editing one-line vector
- Editing multiple-line vector
- Editing loop and repeat count
- Editing pin-based vector

2.2.1.1 Editing One Vector

```
EDIT VEC <label> <vec#>@R<#>@<data>
```

Where `vec#` is the vector position in the pattern, `R<#>` is the repeat count for this vector line, and `<data>` is vector data in order of pins which can be determined by `GET PAT SIGNAL_LIST`.

In x1-mode, it is written as for 5 pins “11111”, 5 data char is mapped to 5 pins;

in x-mode, it is written as, e.g. x3-mode, “111,000,111,010,111”, 5 data separated by “,” mapped to 5 pins.

2.2.1.2 Editing Multiple Vectors

```
EDIT VEC <label> <vec#>@<data>&+
```

In multiple vector editing, vector repeat is kept unchanged. It is flat vector editing and it only needs to put start vector position.

Additional vector data is appended via the `&` sign.

In x1-mode for 5 pins, it is written as 11111&00000&…

In x5-mode for 5 pins, it is written as 111,000,111,010,111&111,000,111,010,111&…

2.2.1.3 Editing Loop and Repeat Counts

Multiple line Loop can be inserted in the middle of a flat pattern.

```
EDIT VEC <label> <vec#>@R<#>@#<#>
```

Where the first # is at the vector position, the second # is the number of repeats, and the third # is the number of vector lines to be repeated.

Any existing loop structure that doesn't match what the instruction looks for activates an error.

For an existing loop structure that matches the instruction, only loop count is modified.

2.2.2 Edit Pin-Based Vector

```
EDIT VEC <label> <pin>+ <vec#>@<data>&+
```

Where <pin>+ can be one pin or several pins separated by ",".

2.2.3 Insert Vector Data

Occasionally additional vector data can be inserted into the pattern needed. It can be done by using the INSERT command.

```
INSERT VEC [<label>] <vec#>@R<n>@data
INSERT VEC [<label>] <vec#>@data
```

- If label is omitted, the primary active label is used.
- <vec#> is the vector line position.
- <n> is the repeat count of this vector.
- <data> is the vector data.

2.2.4 Delete Vector Data

Also, vector data can be trimmed away with DELETE command:

```
DELETE VEC <label> <ipos>
```

<ipos> is the vector line position.

2.2.5 SPLIT Pattern

This command extracts a portion of an existing pattern from a given multiport burst label and a given port. The new label contains vector data of original pattern starting from cycle <spos> and ending at the cycle <epos> (inclusive).

```
SPLIT OPSEQ SRC=<srclabel> PORT=<> STARTC=<scyc> [STOPC=<ecyc>]
```

STOPC can be omitted.

Note: The split command works also for a loop and one or more single line repeats within the loop. However, it doesn't support multiple lines repeat within a loop.

Note: There is a loop at start cycle or stop cycle and the distance between start cycle and stop cycle are more than 100-line vectors. The process of splitting pattern takes an excessive amount of time to complete. In this case, consider to manually flatten the pattern before the split command is used.

The command returns the newly generated label or labels.

The end cycle "ecyc" is within the same pattern, and the new label is named as:

```
<main_label>_<scyc>_epos
```

The end cycle "ecyc" is across multiple patterns within that port. The newly generated labels are listed as:

```
<main_labela>_<scyc>_max1
<main_labelb>_(max1+1)_(max2)
...
<main_labelm>_(pre_max+1)_(maxm)
<main_labeln>_(maxn+1)_(ecye)
```

Where labela, labelb, labelm and labeln are main label names in the burst pattern.

2.3 Test Pattern and Program Conversion

SmartShell supports pattern conversion for SVF and STIL pattern file. The SVF pattern conversion is done with SmartShell SVF built-in converter and STIL conversion is done via stdl "stilreader" tool from TestInsight. SmartShell server supports two EDA commands: one is ATELoadPattern and the other is ATEStreamPattern. Both commands must have the pattern type specified by the pattern suffix ".svf" or ".stil". Please refer to Appendix A in the Tessent SiliconInsight User's Manual for Tessent Shell from Siemens for details.

Notes:

1) The SVF conversion requires an existing and active (primary) timing for pattern conversion because SVF pattern doesn't contain timing information;
2) The primary timing specification can be single-port and multi-port timing and it must have defined a state char "P" for the JTAG clock pin;
3) The conversion will always proceed even if the SmartShell server issues warning when it finds in the active primary timing specification that more or less pins are defined than what was requested by SVF file.

A spreadsheet-based test program conversion in compliance with stdl "convmgr" tool is also supported in the SmartShell. The spread sheet is in Microsoft xlsx format and this spread sheet defines all what a V93000 test program is needed. The "convmgr" parses this spread sheet file, locates correspondent files such as STIL patterns, and creates an ordinary V93000 test program including testflow, pin configuration, timing and patterns.

2.3.1 CONV Test Pattern

Default STIL conversion setup is defined in config.xml file (see second *2.3.4 Configuration for Conversion*). When different STIL conversion setup file is needed, the user can use the following CONV command:

```
CONV STILREADER SETUP=<setup file>
```

Where <setup file> is the STIL conversion setup file for "stilreader". This file must be accessible from the SmartShell server workstation where the conversion takes place. This command is useful when the setup file is comprehensive script like python script.

When the setup file is a plain text and all STIL conversion options are in the single plan text conversion file, the user can stream the setup file over to the SmartShell server using the following command:

```
CONV STILREADER STREAM SETUP=stil.setup @@ <complete setup file string
with @@ as the new line separation>
```

Sometimes the conversion needs additional default STIL Patterns defined in the setup:

```
default_definitions = ['default stil file.stil', 'default stil file
stil']
```

The following command serves the purpose of streaming these default STIL pattern to the SmartShell server so the conversion can succeed:

```
CONV STILREADER STREAM DEFAULTPAT=<default stil file>.stil @@
<complete default STIL pattern string with @@ as the new line
separation>
```

2.3.2 CONV Test Program

```
CONV CONVMGR <spread sheet file>
```

Where `<spread sheet file>` is the spread sheet file. The resulting patterns and test suites are added to the current test program. The conversion can occur in a workstation other than the tester workstation where SmartShell server is running.

2.3.3 File Query

The program to be converted needs to reside in the location where the conversion occurs. In order to make it easier for you to find it out which path and file name for the conversion exactly look like. There is an auxiliary command:

```
CONV UNIX=<UNIX command>
```

Where <UNIX command> is the UNIX command such ls, cd, pwd, etc.

2.3.4 Configuration for Conversion

In order to use this feature, enter host, port, conversion tool in the `config.xml` file located in `/opt/hp93000/smartshell/config/` and there are two cases:

- Conversion is in the same tester workstation where the SmartShell server is running
- Conversion is in the workstation other than the tester workstation

2.3.4.1 Same workstation

The figure Fig. 3 shows how to setup the conversion at the same workstation:

```
<?xml version='1.0' standalone='yes'?>
<smartshell type="app">
        <appinfo homepage="testtheory/General/index.htm"/>
        <setups>
                <setup id="ip_port" title="Specify port number the server is listening on" desc="tooltip" display="mai
                        <value>2227</value>
                </setup>
                <setup id="port_range" title="Specify port range the server internally uses" desc="tooltip" display="m
                        <value>2224-2229</value>
                </setup>
                <setup id="log_path" title="Specify the path to save log data" desc="tooltip" display="main" editable=
                        <value>../log</value>
                </setup>
               <setup id="debug" title="Flag to enable logging verbose" desc="tooltip" display="main" editable="1" typ
                        <!--
                                true  : open
                                false : close
                        -->
                        <value>false</value>
                </setup>
                <setup id="reload" title="Flag to enable vector reload when smartshell exits" desc="tooltip" display="
                        <!--
                                true  : open
                                false : close
                        -->
                        <value>true</value>
                </setup>
                <setup id="servertype" title="Specify the server type" desc="tooltip" display="main" editable="1" type
                        <!--
                                server:      this server does both SmartShell operation and conversion operation
                                converter:   this server does only conversion operation
                                host or ip:  the remote server at which the conversion is performed
                                             in this case stilconverter and xlsxconvmgr entries are ignored
                        -->
                        <!-- <value>host</value> -->
                        <value>server</value>
                        <!-- <value>converter</value> -->
                </setup>
                <setup id="stilconverter" title="Specify stil conversion tool with the full path" desc="tooltip" displ
                        <value>/opt/93000/stdl/bin/stilreader</value>
                        <value>-setup</value>
                        <value>/opt/hp93000/smartshell/config/stil.setup</value>
                </setup>
                <setup id="xlsxconvmgr" title="Specify convmgr tool with the full path" desc="tooltip" display="main"
                        <value>/opt/93000/stdl/bin/convmgr</value>
                </setup>
        </setups>
</smartshell>
```

Fig. 3: "`config.xml`" for the conversion at the same workstation

The `config.xml` file puts the key word `server` in section `stilserver` and specifies the name and the path of the `convmgr` tool for the conversion.

2.3.4.2 Different workstation

In this case, the SmartShell server need to be run both on the tester workstation and on the conversion workstation and their `config.xml` files should look like this:

```
<?xml version='1.0' standalone='yes'?>
<smartshell type="app">
        <appinfo homepage="testtheory/General/index.htm"/>
        <setups>
                <setup id="ip_port" title="Specify port number the server is listening on" desc="tooltip" display="mai
                        <value>2227</value>
                </setup>
                <setup id="port_range" title="Specify port range the server internally uses" desc="tooltip" display="m
                        <value>2224-2229</value>
                </setup>
                <setup id="log_path" title="Specify the path to save log data" desc="tooltip" display="main" editable=
                        <value>../log</value>
                </setup>
               <setup id="debug" title="Flag to enable logging verbose" desc="tooltip" display="main" editable="1" typ
                        <!--
                                true  : open
                                false : close
                        -->
                        <value>false</value>
                </setup>
                <setup id="reload" title="Flag to enable vector reload when smartshell exits" desc="tooltip" display="
                        <!--
                                true  : open
                                false : close
                        -->
                        <value>true</value>
                </setup>
                <setup id="servertype" title="Specify the server type" desc="tooltip" display="main" editable="1" type
                        <!--
                                server:      this server does both SmartShell operation and conversion operation
                                converter:   this server does only conversion operation
                                host or ip:  the remote server at which the conversion is performed
                                             in this case stilconverter and xlsxconvmgr entries are ignored
                        -->
                        <value>workstation4conversion</value>
                        <!-- <value>server</value> -->
                        <!-- <value>converter</value> -->
                </setup>
                <setup id="stilconverter" title="Specify stil conversion tool with the full path" desc="tooltip" displ
                        <value>/opt/93000/stdl/bin/stilreader</value>
                        <value>-setup</value>
                        <value>/opt/hp93000/smartshell/config/stil.setup</value>
                </setup>
                <setup id="xlsxconvmgr" title="Specify convmgr tool with the full path" desc="tooltip" display="main"
                        <value>/opt/93000/stdl/bin/convmgr</value>
                </setup>
        </setups>
</smartshell>
```

Fig. 4: "`config.xml`" in the tester workstation

The `config.xml` in the tester workstation needs to provide the `<conversion_workstation_hostname>` in section `stilserver`.

```
<?xml version='1.0' standalone='yes'?>
<smartshell type="app">
        <appinfo homepage="testtheory/General/index.htm"/>
        <setups>
                <setup id="ip_port" title="Specify port number the server is listening on" desc="tooltip" display="mai
                        <value>2227</value>
                </setup>
                <setup id="port_range" title="Specify port range the server internally uses" desc="tooltip" display="m
                        <value>2224-2229</value>
                </setup>
                <setup id="log_path" title="Specify the path to save log data" desc="tooltip" display="main" editable=
                        <value>../log</value>
                </setup>
               <setup id="debug" title="Flag to enable logging verbose" desc="tooltip" display="main" editable="1" typ
                        <!--
                                true  : open
                                false : close
                        -->
                        <value>false</value>
                </setup>
                <setup id="reload" title="Flag to enable vector reload when smartshell exits" desc="tooltip" display="
                        <!--
                                true  : open
                                false : close
                        -->
                        <value>true</value>
                </setup>
                <setup id="servertype" title="Specify the server type" desc="tooltip" display="main" editable="1" type
                        <!--
                                server:      this server does both SmartShell operation and conversion operation
                                converter:   this server does only conversion operation
                                host or ip:  the remote server at which the conversion is performed
                                             in this case stilconverter and xlsxconvmgr entries are ignored
                        -->
                        <!-- <value>host</value> -->
                        <!-- <value>server</value> -->
                        <value>converter</value>
                </setup>
                <setup id="stilconverter" title="Specify stil conversion tool with the full path" desc="tooltip" displ
                        <value>/opt/93000/stdl/bin/stilreader</value>
                        <value>-setup</value>
                        <value>/opt/hp93000/smartshell/config/stil.setup</value>
                </setup>
                <setup id="xlsxconvmgr" title="Specify convmgr tool with the full path" desc="tooltip" display="main"
                        <value>/opt/93000/stdl/bin/convmgr</value>
                </setup>
        </setups>
</smartshell>
```

Fig. 5: "`config.xml`" in the conversion workstation

The `config.xml` in the conversion workstation needs to put the port number `2229` in `ip_port` section and put the key word `converter` in section `stilserver`.

2.4 Temporary Test suite

A temporary test suite can be defined on the fly and used immediate afterwards:

`SET SUITE [<suite>] TM_LINK=<tm> SPECS=<specs> OPSEQ=<label>`

When `suite` is not provided, then a temporary test suite is created internally.

`tm` is the name of the test method and follows the path shown in Test Method Selection Window under SmarTest 7.

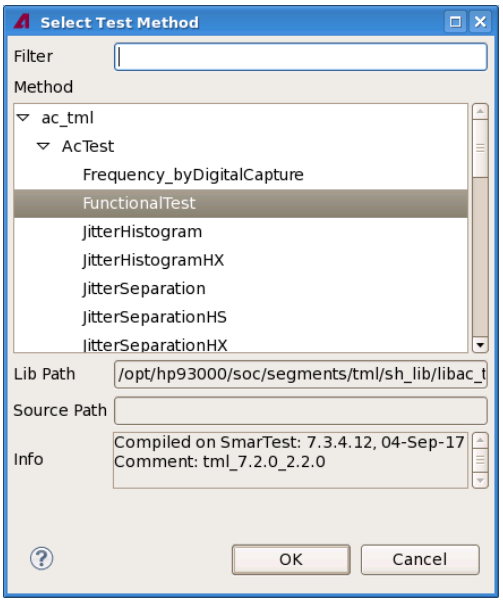


Fig. 6: Select Test Method Pop-up Window

`specs` is composed of `levelset` and `timingset`.

- `levelset` is given as <eqn#>, <spec#>, <set#>.
- `timingset` is given either as <eqn#>, <spec#>, <set#> for set based timing, or as <timing_specification>@<set#>+ for multiport-based timing.

`label` is the burst label.

2.4.1 SET Values to TestMethod Parameters

Each of the TestMethod parameters can be assigned to a value or strings using the following instruction:

```
SET SUITE [<suite_name>] TM_VALUE <"parameter=value"> +
```

`parameter=value` must be quoted.

2.4.2 SET Flow Variable

The same is true also for testflow variable:

```
SET FLOW VAR:  SET FLOW [<flow>] VAR INT/DOUBLE/STRING "<var>=<value>"
```

2.4.3 SET Suite Flag

```
SET SUITE FLAG: SET SUITE <suite_name> FLAG BYPASS <flag_value>
```

2.5 Test Setup Modification

2.5.1 Set Timing Edge and Level Threshold

This command directly operates settings on the tester hardware:

```
SET LEV_DPS: SET LEV_DPS <pin>=<value>
```

```
SET LEV:  SET LEV <pin>,+ <levelkeyword>=<value>+
SET EDGE:  SET EDGE <pin>,+ (<edge>=<value>+)
```

2.5.2 Set Spec Variable Value

Values of spec variable can be changed via the following command:

```
SET SPEC VAR:  SET SPEC VAR TIM/LEV <var>=<value>
```

2.6 Query Command

2.6.1 GET FLOW and SUITE Name and Variable

To query the loaded testflow information, the following group of commands are used:

```
GET FLOW SUITE_LIST:  GET FLOW [<flow>] SUITE_LIST
GET FLOW VAR:  GET FLOW [<flow>] VAR [TYPE] <var>
GET FLOW VAR_LIST:  GET FLOW [<flow>] VAR_LIST [TYPE]
GET SUITE TM_PARAM:  GET SUITE [<suite_name>] TM_PARAM
GET SUITE TM_PARAM_WITH_VALUE:  GET SUITE [<suite_name>]
TM_PARAM_WITH_VALUE
GET SUITE TM_VALUE:  GET SUITE [<test suite>] TM_VALUE "<var>" +
GET SUITE FLAG: GET SUITE <suite_name> FLAG BYPASS
```

Details can be found in the SmartShell section *4.12*.

2.6.2 GET TIMING EDGE and LEVEL Settings

To query the exact timing and level setting, the following commands deal with them:

```
GET EDGE:  GET EDGE <pin>
GET LEV:  GET LEV <pin>
```

2.6.3 GET PAT for Pattern Sequencing

The burst labels or operating sequences and individual patterns can be queried back:

```
GET PAT PRIMARY:  GET PAT PRIMARY
GET PAT LABEL_LIST MULTI_PORT:  GET PAT LABEL_LIST MULTI_PORT
GET PAT LABEL_LIST SINGLE_PORT:  GET PAT LABEL_LIST SINGLE_PORT
GET PAT LABEL_LIST:  GET PAT <label> LABEL_LIST <port>
```

GET Signal list and Port/Group Information

```
GET PAT PORT_NAME:  GET PAT <label> PORT_NAME <pin>
GET PAT SIGNAL_LIST:  GET PAT <label> SIGNAL_LIST
```

Get Total Vector and Total Cycle Counts

```
GET PAT TOTAL_CYC:  GET PAT <label> TOTAL_CYC
GET PAT TOTAL_VEC:  GET PAT <label> TOTAL_VEC
```

Get Vector Data and Vector Data Per Pin

```
GET PAT VEC SIGNAL:  GET PAT <label> VEC <pin> <start> <stop>
GET PAT VEC:  GET PAT <label> VEC <start> [STATEMAP_TABLE]
```

Get Pattern Sequencing Setting from Cycle/Vector Number

```
GET PAT VNO CYC:  GET PAT <label> VNO <port> CYC=<ipos>
GET PAT VNO TAG:  GET PAT <label> VNO <port> TAG=<tag>
```

2.6.4 GET SIGNAL LIST

Signal list can be defined as a group or under SmarTest 7 with a port

```
GET SIGNAL_LIST GROUP:  GET SIGNAL_LIST GROUP <group>
GET SIGNAL_LIST PORT:  GET SIGNAL_LIST PORT <port>
```

2.6.5 GET SPEC for Level and Timing Setups

V93000 related timing and level specification and its settings can be queried:

```
GET SPEC LEV EQN:  GET SPEC [<eqn_number/eqn_name>/ALL/ACTIVE] LEV [EQN]
GET SPEC PERIOD:  GET SPEC [<specification>] PERIOD <port_name/pin_name>
GET SPEC TIM EQN:  GET SPEC [<eqn_number/eqn_name>/ALL/ACTIVE] TIM EQN
GET SPEC TIM:  GET SPEC [ALL/ACTIVE] TIM
GET SPEC VAR:  GET SPEC [<specification>] VAR [TYPE] <>
GET SPEC VAR_LIST:  GET SPEC [<specification>] VAR_LIST [TIM/LEV]
```

2.6.6 GET XMODE for Pin X-mode Setting

A device cycle can use multiple timing edges called xmode and can be queried back:

```
GET XMODE:  GET XMODE [<specification>] <pin_name>
```

2.6.7 GET SmarTest, SmartShell and Test Program Information

SmartShell must always be connected to one SmarTest session. The session information can be queried back:

```
GET SESSION EVENT:  GET SESSION EVENT <session_id_key>
GET SESSION STATUS:  GET SESSION STATUS <session_id_key>
GET SMARTEST REV:  GET SMARTEST REV
GET SMARTSHELL REV:  GET SMARTSHELL REV
```

2.6.8 GET Test Program

```
GET TP_PATH:  GET TP_PATH
```

2.6.9 GET UI Report Message

```
GET UI_REPORT:  GET UI_REPORT

GET EVENT_LOG:  GET EVENT_LOG
```

2.7 Execution

2.7.1 Set Error Limit

SET ERR_LIMIT: SET ERR_LIMIT <number>

2.7.2 Mask and Unmask Pin

SET MASK: SET MASK <pin>+
SET UNMASK: SET UNMASK <pin>+

2.7.3 Flow Execution

EXE FLOW executes the entire testflow.

EXE TO_SUITE executes the flow from the beginning until a specified test suite.

2.7.4 Suite Execution

EXE SUITE executes the temporary test suite or a given test suite.

2.7.5 Pattern/OpSeq Execution

The EXE command can execute either main label or burst label or operating sequence.

The followings are the options used for the call:

PAT	: must present none of FAILC exists
FAILC	: failing cycle
FAILC_CNT	: total fail count of all pins
FAILC_PIN	: failing count per pin
TAG	: get vector comment text at failing cycle
MAIN=<labels>	: list of main label separated by ","
OPSEQ=<labels>	: burst label or operating sequence
SIGNAL=<pinlist>	: pin list from which the result is retrieved, separated by ","
STARTC=<spos>	: cycle number start from the beginning of the pattern execution
NO_OF_CYCLES=<n>	: number of fails

The result can be either pass or fail.

The failing cycle information is needed. The format is like this:

```
ATE_FAILC  cycle#     pin1       expected   result
ATE_FAILC  14958      gpio_78    HHLLL      LLLLL
```

3 Rules of Return Message

- ERROR – Normal error message.
    - ERROR: FATAL_ERROR: – Need to restart SmartShell or SmarTest.
    - ERROR: SYNTAX_ERROR: – The SmartShell command syntax is incorrect, the possible commands are listed below.
- WARNING – Normal warning message.
- Normal result string.

4 Command References

4.1 System Commands

4.1.1 INIT

Description	**The "INIT" command starts a new SmarTest session with the specified settings and executes the specified test program. Depending on the complexity of the project, this might take some time.**
Syntax	INIT SESSION=<session_id> TESTPROGRAM=<test_program_name> FLOW=<testflow_name> MODEL=<model_file_name> MODE=<mode>
Parameters	session_id: The ID for the new session. project_name: The name of the SmarTest project to be used for the new session. test_program_name: The name of the test program file to be used for the new session, including the relative path in the project folder. testflow_name: The name of the file. model_file_name: The absolute path of the model file to be used for the new session. mode: The operating mode of SmarTest, online or offline.
Return Value	pass
Return Error	Condition: The model file does not exist. Return: ERROR: The <model-file-name> does not exist. Condition: The test program does not exist. Return: ERROR: The <test-program-name> does not exist. Condition: The testflow does not exist. Return: ERROR: The <test-flow-name> does not exist. Condition: The session ID is already used and connected with different settings. Return: ERROR: <The session-id> is already connected .
Example	>>> INIT SESSION=test FLOW=/home/user/device/testflow/de0_nano_soc_2nd.flw MODEL=/etc/opt/hp93000/soc/tester.model MODE=online <<<pass

4.1.2 CHANGE DEVICE

Description	**The "CHANGE DEVICE" command changes the device and load the relevant setup files .**
Syntax	CHANGE DEVICE PATH=<device-path> FLOW=<test-flow-name>
Parameters	device-path: The directory of the device to load. test-flow-name: The name of the test flow to use.
Return Value	pass
Return Error	Condition: The device path does not exist

	Return: ERROR: The <device-path> does not exist Condition: The test flow does not exist Return: ERROR: The <test-flow-name> does not exist
Example	>>> CHANGE DEVICE PATH=/home/user/device FLOW=test.flw <<<pass

4.1.3 RESET

Description	**The "RESET" command clears all internal cache in the SmartShell server. It is needed prior the use of "USE SESSION" to connect a new SmarTest session.**
Syntax	RESET
Parameters	None
Return Value	pass
Return Error	None
Example	>>>RESET <<<pass

4.1.4 LOAD

Description	**The "LOAD" command loads a specified testflow, then executes the flow**
Syntax	LOAD [TF=<testflow_name>] [TP=<testprogram_name>]
Parameters	testflow_name or testprogram_name
Return Value	pass
Return Error	**Condition:** the testflow name does not exist or can not be loaded properly **Return:** ERROR: Failed to load the testflow. **Condition:** the testprogram file does not exist **Return:** ERROR: The prog file does not exit. **Condition:** the testprogram can not be loaded properly **Return:** ERROR: Failed to load the testprogram.
Example	>>> LOAD TF=SmartShellDemo.flw Analyzing testflow SmartShellDemo.flw... Analyzing pin configuration SmartShellDemo.cfg... Analyzing timing SmartShellDemo.tim... pass

4.1.5 RELOAD

Description	**The "RELOAD" command reloads some or all of the test program components such as pin, timing, level, vector patterns or testflow. This command works only after the test program has been fully loaded.** **If setups are removed from the system, "RELOAD" command won't help. This situation occurs when the "RELOAD PIN" command is issued. This "RELOAD PIN" command removes the level, timing, setup files etc. from the system. In**

	order to load all setup files again, use "LOAD" command to explicitly reload all the other setup components into the system
Syntax	RELOAD [ALL] [LEVEL] [TIMING] [VECTOR] [TESTFLOW]
Parameters	ALL: reload all settings PIN: reload pin config LEVEL: reload levels settings TIMING: reload timing settings VECTOR: reload vector patterns TESTFLOW: reload test flow
Return Value	pass
Return Error	**Condition:** the parameter is wrong **Return:** WARNING: Unknown parameter xxx.
Example	>>>RELOAD ALL <<<DFT_RESET_ATE TESTFLOW LEVEL TIMING VECTOR <<<pass >>>RELOAD LEVEL <<<pass

4.1.6 SAVE SETUP

Description	**The "SAVE" command saves the setup files of the given type.**
Syntax	SAVE SETUP [ALL] [PIN] [LEVEL] [TIMING] [VECTOR] [TESTFLOW]
Parameters	ALL: refresh all settings PIN: refresh pin config LEVEL: refresh levels settings TIMING: refresh timing settings VECTOR: refresh vector patterns TESTFLOW: refresh test flow
Return Value	pass
Return Error	None
Example	>>>SAVE SETUP LEVEL <<<pass

4.1.7 UPDATE SETUP

Description	**The "UPDATE SETUP" command re-analyzes the setup files to keep the setup information updated in the SmartShell server.**
Syntax	UPDATE SETUP
Parameters	None
Return Value	Pass
Return Error	None
Example	>>>UPDATE SETUP <<<pass

4.1.8 GET SMARTSHELL REV

Description	**The "`SET SMARTSHELL REV`" command returns SmartShell revision number**
Syntax	`GET SMARTSHELL REV`
Parameters	None
Return Value	Currently used revision number of SmartShell
Return Error	None
Example	`>>>GET SMARTSHELL REV` `<<<SmartShell Revision v3.4.4.2-20190816`

4.1.9 GET SMARTEST REV

Description	**The "`GET SMARTEST REV`" command returns SmarTest revision**
Syntax	`GET SMARTEST REV`
Parameters	None
Return Value	Currently used revision number of SmarTest.
Return Error	None
Example	`>>>GET SMARTEST REV` `<<<SmarTest Revision 7.4.0.2 (T), 64bit`

4.1.10 GET CUR_SESSION

Description	**The "`GET CUR_SESSION`" command return the connected SmarTest session**
Syntax	`GET CUR_SESSION`
Parameters	None
Return Value	The connected SmarTest session
Return Error	None
Example	`>>>GET CUR_SESSION` `<<<Session 1: ID=smartshell_demo`

4.1.11 GET ALL_SESSIONS

Description	**The "`GET ALL SESSIONS`" command lists all the running SmarTest sessions**
Syntax	`GET ALL_SESSIONS`
Parameters	None
Return Value	All the running SmarTest sessions
Return Error	None
Example	`>>>GET ALL_SESSIONS` `<<<Session id`

4.1.12 GET SESSION MODE

Description	**The "`GET SESSION MODE`" command returns SmarTest mode**

Syntax	GET SESSION MODE
Parameters	None
Return Value	SmarTest mode
Return Error	**None**
Example	>>>GET SESSION MODE <<<OFFLINE

4.1.13 GET SESSION STATUS

Description	**The "GET SESSION STATUS" command returns SmarTest status**
Syntax	GET SESSION STATUS <session_id>
Parameters	session_id: SmarTest session ID which is defined by DISPLAY_HP83000
Return Value	SmarTest is ready
Return Error	**Condition**: the session does not exist. **Return**: ERROR: No SmarTest session session_id found. **Condition**: the setup files are not loaded properly **Return**: WARNING: Please check the setup files are already loaded properly.
Example	>>>GET SESSION STATUS smartshell_demo <<<sessionID = smartshell_demo <<<SmarTest execution status for SmarTest session "smartshell_demo": READY <<<mode: OFFLINE

4.1.14 GET DOCK_STATUS

Description	**The "GET DOCK_STATUS" command gets the status if the DUT interface board is docked mechanically to the tester head.**
Syntax	GET DOCK_STATUS
Parameters	None
Return Value	DUT_DOCKED "FALSE" DUT_DOCKED "TRUE"
Return Error	None
Example	>>>GET DOCK_STATUS <<<DUT_DOCKED "FALSE"

4.1.15 GET CONNECT_STATUS

Description	**The "GET CONNECT_STATUS" command checks if pin electronic relays are connected**
Syntax	GET CONNECT_STATUS
Parameters	None
Return Value	CONNECTED or NOT CONNECTED OR PARTIAL CONNECTED. DETAILS is provided, the it returns connection status for all pins.

Return Error	None
Example	>>>GET CONNECT_STATUS <<<CONNECTED "FALSE" <<<CONNECTED PINS:BADD0,BADD1 <<<DISCONNECTED PINS:BADD2,BADD3

4.1.16 GET TP_PATH

Description	**The "GET TP_PATH" command returns the test program directory path**
Syntax	GET TP_PATH
Parameters	None
Return Value	Device path
Return Error	None
Example	>>>GET TP_PATH <<<output

4.1.17 GET TP LIST

Description	**The "GET TP LIST" command lists all test programs in the current device path**
Syntax	GET TP LIST
Parameters	None
Return Value	List of test programs if any in the current device path
Return Error	None
Example	>>>GET TP LIST <<<device path : /home/demo/xxxx/xxxx

4.1.18 GET TF LIST

Description	**The "GET TF LIST" command lists all testflows in the current device path**
Syntax	GET TF LIST
Parameters	None
Return Value	A list of testflows in current device path
Return Error	None
Example	>>>GET TF LIST <<<smartshell_demo.flw <<<smartshell_demo.flw.bak

4.1.19 GET LOG_PATH

Description	**The "GET LOG_PATH" command returns the temporary folder for storing session data**
Syntax	GET LOG_PATH
Parameters	None
Return Value	Temporary storage folder

Return Error	None
Example	>>>GET LOG_PATH <<< /opt/hp93000/smartshell64-3.4.5.0_el7/bin/../log/sms.GnyZw0n9/

4.1.20 GET UI_REPORT

Description	**The "GET UI_REPORT" command obtains the actual UI report window content**
Syntax	GET UI_REPORT [SUITE/OPSEQ]
Parameters	SUITE: results from the site in the UI report window OPSEQ: results referring to pattern burst execution if any in the UI report window
Return Value	Complete UI report text
Return Error	**Condition**: can not open uireport.dump **Return**: ERROR: Failed to get the UI info
Example	>>>GET UI_REPORT <<<xxxxxxxxxxxx <<<Info from UI Report <<<xxxxxxxxxxxxxx

4.1.21 GET EVENT_LOG

Description	**The "GET EVENT_LOG" command obtains the actual event log when executing a test suite. This can be used after command "EXE SUITE".**
Syntax	GET EVENT_LOG
Parameters	None
Return Value	Event log after test suite execution
Return Error	**None**
Example	>>>GET EVENT_LOG <<<xxxxxxxxxxxx <<<Info from event log <<<xxxxxxxxxxxxx

4.1.22 USE SESSION

Description	**The "USE SESSION" command selects from one of the existing SmarTest sessions for connection and operates on this connected session**
Pre-condition	You can get all the running SmarTest sessions with command “GET ALL_SESSIONS”, then use “USE SESSION” to connect one session.
Syntax	USE SESSION=<session_id_key>
Parameters	session_id_key: SmarTest session ID which is defined by DISPLAY_HP83000
Return Value	IDLE: SmarTest starts and test program is set, but setups have not been downloaded READY: SmarTest starts and test program is loaded NOT READY: SmarTest starts but no test program is set

	FAILED: system errors of various kinds
Return Error	**Condition:** command syntax error **Return:** ERROR: Unsupported command syntax USE xxx **Condition:** the session does not ready. **Return:** ERROR: Failed to connect to SmarTest session <SessionID> **Condition:** the session does not exist. **Return:** ERROR: SmarTest session <SessionID> does not exist
Example	>>>USE SESSION=smartshell_demo <<<sessionID = smartshell_demo <<<SmarTest execution status for SmarTest session "smartshell_demo": READY

4.1.23 CONNECT_DUT

Description	**The "CONNECT_DUT" command sets pin electronic relays connected which behaves exactly the same as click the connect button in SmarTest Work Center menu.**
Syntax	CONNECT_DUT
Parameters	None
Return Value	pass no error;
Return Error	ERROR: <any reason from tester>
Example	>>>CONNECT <<<pass

4.1.24 DISCONNECT_DUT

Description	**The "DISCONNET_DUT" command disconnects the pin electronic relays, which behaves exactly the same as clicking the disconnect button in the SmarTest Work Center menu**
Syntax	DISCONNECT_DUT
Parameters	None
Return Value	pass no error;
Return Error	ERROR: <any reason from tester>
Example	>>>DISCONNECT_DUT <<<pass

4.1.25 CLOSE_SOCKET

Description	**The "CLOSE_SOCKET" command quits the operation. Now the server is waiting for new connection by client.**
Syntax	CLOSE_SOCKET
Parameters	None
Return Value	pass no error
Return Error	ERROR: <any reason from tester>
Example	>>>CLOSE_SOCKET <<<pass

4.1.26 CLOSE_SMT

Description	**The "CLOSE_SMT" command closes the SmarTest of a given session ID**
Syntax	CLOSE_SMT [id]
Parameters	id: Optional. It is given the specified session id is closed.
Return Value	pass no error
Return Error	ERROR: <any reason from tester>
Example	>>>CLOSE_SMT xyz <<<pass

4.1.27 read

Description	**The "read" command reads all supported commands and executes them**
Syntax	read <SmartShell command script>
Parameters	None
Return Value	Execution messages for each of individual command
Return Error	Error messages of each command
Example	"smartshell_command_file" contains the following SmartShell command list: RELOAD VECTOR SET PAT aname1 CONSTR example_1 SET SIGNAL_LIST IO_000 IO_002 IO_004 IO_006 IO_008 IO_010 IO_012 IO_014 IO_016 IO_018 IO_020 SET VEC R1 00000000000 SET PAT_END aname1 STOP SET PAT aname2 CONSTR example_2 SET SIGNAL_LIST IO_001 IO_003 IO_005 IO_007 IO_009 IO_011 IO_013 IO_015 IO_017 IO_019 IO_021 SET VEC R1 HHHHHHHHHHH // there is an error SET VEC R1 HHHHHHHHHHH SET VEC R1 HHHHHHHHHHH // there is another error SET PAT_END aname2 STOP SET PAT aname3 CONSTR example_3 SET SIGNAL_LIST tck tdi tdo tms trst_n SET VEC R3 00000 SET PAT_END aname3 STOP SET OPSEQ seq_group_name1 SEQ PORT=example_1 LABEL_LIST=aname1 SET OPSEQ seq_group_name2 SEQ PORT=example_2 LABEL_LIST=aname2 SET OPSEQ seq_group_name3 SEQ PORT=example_3 LABEL_LIST=aname3 SET OPSEQ paral_group_name PAR LABEL_LIST=seq_group_name1 seq_group_name2 seq_group_name3 >>> read <path_to_file>/smartshell_command_file

4.1.28 HELP

Description	**The "HELP" command lists all supported commands or the sub commands**
Syntax	HELP ALL/<command>
Parameters	None
Return Value	information of the command list or individual command
Return Error	None

Example	>>> HELP GET <<<GET ALL_SESSIONS: GET ALL_SESSIONS GET BOARD_CONF: GET BOARD_CONF PIN/PORT/GROUP GET CONNECT_STATUS: GET CONNECT_STATUS GET DOCK_STATUS: GET DOCK_STATUS GET DVCMAP: GET DVCMAP <pin_name>…….

4.2 Pattern Generation

Note: Basic Pattern Generation, commands should be used following the steps as below.

```
SET PAT CONSTR  – define the label name and relevant port
SET SIGNAL_LIST – define the pins used in the label
SET VEC         - define the vector data
SET VEC_END     - define the end of the label
```

4.2.1 SET PAT CONSTR

Description	**The "SET PAT CONSTR" command sets the name of the pattern <label> for the following pattern generation commands including:** **SET SIGNAL_LIST** **SET STATEMAP** **SET VEC** **SET TOGGLE** **SET NOP** **SET VEC_END** **SET OPSEQ**
Syntax	SET PAT <label> CONSTR <port>
Parameters	label: the name of the main label port : port name can be provided
Return Value	pass no error
Return Error	None
Example	>>>SET PAT new_label CONSTR jtag_port <<<pass

4.2.2 SET SIGNAL_LIST

Description	**The "SET SIGNAL_LIST" command is a member command within "SET PAT CONSTR" command group and it provides the complete pin list for the "SET VEC" command.** **The Order of pin list is the order of vector data.**
Syntax	SET SIGNAL_LIST <pin>+
Parameters	<pin>+: list of pin name for the command group initialized by SET PAT CONSTR. It can be separated by either blank " " or comma ","
Return Value	pass no error
Return Error	**Condition**: the specified pins don't belong to a port **Return**: ERROR: (CODE_1): can not find port from V93000 setup base on pins of DFT_SIGNALS.
Example	>>>SET SIGNAL_LIST pin1 pin2 <<<pass

4.2.3 SET STATEMAP

Description	**The "SET STATEMAP" command is a member command within "SET PAT CONSTR" command group and it defines the <statemap> name used for the vector data conversion.** **The STATEMAP is defined in timing wavetable block** PINS CLOCK 0 "{ d1:1 d2:FON }" P 1 "{ d1:1 }" 1 2 "{ d1:0 }" 0 3 "{ d1:Z }" Z 4 "{ r1:X }" X brk "" STATEMAP #physSeq statechar period selector 0 P x1 project plltst_d_jtag_p_12/P TS1/P 1 1 x1 project plltst_d_jtag_p_12/1 TS1/1 2 0 x1 project plltst_d_jtag_p_12/0 TS1/0 3 Z x1 project plltst_d_jtag_p_12/Z TS1/Z 4 X x1 project plltst_d_jtag_p_12/X TS1/X **where "project" is a key word to tell the server, the following strings are the mapping name of statemap used for pattern generation. In the example "plltst_d_jtag_p_12" is the name that needs to be provided in SET STATEMAP command like this:** **SET STATEMAP plltst_d_jtag_p_12** **There can be multiple statemap names after the "project" keyword like this:** **STATEMAP** **0 P x1 project xyz abc ...** **"abc" should be used, the "SET STATEMAP" command shall be like this:** **SET STATEMAP xyz**
Syntax	SET STATEMAP <statemap_name>
Parameters	statemap_name: name of statemap key for the command group initialized by "SET PAT CONSTR"
Return Value	pass no error
Return Error	None
Example	>>>SET STATEMAP xyz <<<pass

4.2.4 SET VEC

Description	**The "SET VEC" command is a member command within "SET PAT CONSTR" command group and it defines vector data and repeat count.**
Syntax	SET VEC R<n> <data> [// comment string]

Parameters	R<n>: R states for the repeat keyword and n is the repeat counts data: is the vector data in x1 format which must have equal number of chars as the number of pins defined by “SET SIGNAL_LIST” command."
Return Value	pass no error
Return Error	None
Example	>>>SET VEC R1 0000 <<<pass >>>SET VEC R1 0000 // command write header <<<pass

4.2.5 SET TOGGLE

Description	**The "SET TOGGLE" command is a member command within the "SET PAT CONSTR" command group and it tells the server to change the data just at the specified position**
Syntax	SET TOGGLE R<n> <pin_index>@<data>+ All bits of data are kept the same as it is provided by the previous SET VEC command except vector data at <pin_index> position needs to be updated.
Parameters	R<n> : R states for the repeat keyword and n is the repeat counts pin_index: the pin index number defined by “SET SIGNAL_LIST” command data : the vector data in x1 format
Return Value	pass no error
Return Error	None
Example	>>>SET TOGGLE R10 3@L 4@H <<<pass

4.2.6 SET NOP

Description	**The "SET NOP" command is a member command within "SET PAT CONSTR" command group and it just repeats the previous vector for <n> times.**
Syntax	SET NOP R<n>
Parameters	R<n>: R states for the repeat keyword and n is the repeat counts
Return Value	pass no error
Return Error	None
Example	>>>SET NOP R10 <<<pass

4.2.7 SET VEC_END

Description	**The "SET VEC_END" command describes that the pattern data transfer ends temporarily and the transfer can be resumed later. In order to resume the data transfer, issue "SET PAT CONSTR" with the same label again, then this command can continue.**
Syntax	SET VEC_END
Parameters	None
Return Value	Pass

Return Error	None
Example	>>>SET VEC_END <<<pass

4.2.8 SET PAT_END

Description	**The "SET PAT_END" command describes the end of a pattern data transfer and the pattern is ready for conversion.**
Syntax	SET PAT_END <label>+
Parameters	The pattern name defined by SET PAT CONSTR
Return Value	pass no error
Return Error	Condition: the pattern can not be converted Return: ERROR: Fail to generate V93000 binary vector file xxxx. ERROR: Pin 'pin1', state char 'L' is used but not defined in V93000 timing setup
Example	>>>SET PAT_END pat1 pat2 <<<pass

4.2.9 SET OPSEQ

Description	**The "SET OPSEQ SEQ" command creates an one-port burst label <seqlabel> with multiple patterns <main label>+.** **The "SET OPSEQ PAR" command creates a multiport burst label <parlabel> with multiple one-port burst labels <seqlabel>+ created by "SET OPSEQ SEQ" command.**
Syntax	SET OPSEQ <seq_burst_label> SEQ PORT=<port name> LABEL_LIST=<main label list>+ SET OPSEQ <par_burst_label> PAR LABEL_LIST=<seq_burst_label>+
Parameters	seq_burst_label : main label main label list : list of main label under a given pin group or port seq burst label : sequential burst label under a given pin group or port par_burst_label : multi-port burst label name"
Return Value	pass no error
Return Error	Condition: the pattern is not defined in SmarTest Return: ERROR: Failed to generate V93000 vector for port portName, label szLabel Condition: the given burst name is too long Return: WARNING: The burst length is too long, Max length is 128, change the burst name to tmpMPB
Example	>>>SET OPSEQ jtagport_burst_label SEQ PORT=jtat_port LABEL_LIST=pat1 pat2 <<<pass >>>SET OPSEQ multiport_burst PAR LABEL_LIST=jtagport_burst_label otherport_burst_label <<<pass

4.3 Pattern Conversion with 3rd Party Conversion Tool

4.3.1 CONV STILREADER SETUP

Description	**The "CONV STILREADER SETUP" command specifies a user-defined STIL setup file for STIL conversion. Several pre-conditions need to be fulfilled when using this approach:** **1) The STIL setup file and all associated setup files including default STIL pattern files must be accessible from the SmartShell server;** **2) Environment variables required by STIL setup file must be set prior to the launch of the SmartShell server.** **This command can be ideally used in a situation when the STIL setup file is a script**
Syntax	CONV STILREADER SETUP=<stil setup file>
Parameters	<stil setup file>: the STIL setup file with path if needed
Return Value	pass if no error
Return Error	ERROR: if file is not found
Example	>>> CONV STILREADER SETUP= /home/demo/stil_folder/atpg/ategen_STIL_ATPG.setup <<<pass

4.3.2 CONV STILREADER STREAM SETUP

Description	**The "CONV STILREADER STREAM SETUP" command streams the complete STIL setup file to the SmartShell server. This command is used for situation when the STIL setup file is a plain text file and all options are defined in it.**
Syntax	CONV STILREADER STREAM SETUP=<stil setup file> @@ <complete setup file string with @@ as the new line separation>
Parameters	<stil setup file>: an arbitrary name
Return Value	Pass
Return Error	None

4.3.3 CONV STILREADER STREAM DEFAULTPAT

Description	**The "CONV STILREADER STREAM DEFAULTPAT" streams the default STIL pattern defined in the STIL setup file to the SmartShell. These default STIL patterns must appear with the keyword:** **default_definitions = ['default stil file.stil', 'default stil file stil']**
Syntax	CONV STILREADER STREAM DEFAULTPAT=<default stil file>.stil @@ <complete default STIL pattern string with @@ as the new line separation>
Parameters	<default stil file>: name of the STIL pattern file used in the STIL setup file
Return Value	pass
Return Error	None

4.3.4 CONV STILREADER TIM

Description	**The "CONV STILREADER TIM" set the existing timing to the SmartShell. The timing is used for the generated patterns' execution.**
Syntax	`CONV STILREADER TIM=<[eqnno,specnu,setno] or [<specification>@<setno,+>]>`
Parameters	*SETS based level or timing:* `eqnno:` the number of the equation defined in level or timing equation setup `specno:` the number of the spec defined by SpecTool `setno:` the number of either as LEVELSET or TIMINGSET *Multi-port based timing:* `Specification:` defined in SpecTool `setno:` the number of TIMINGSET for each port only one setno if all ports use the same TIMINGSET #
Return Value	pass
Return Error	None

Example	`CONV STILREADER TIM=1,1,1`
	`CONV STILREADER TIM=multiport_timing_specs@1,1,2 or`
	If all setno are the same then
	`CONV STILREADER TIM=multiport_timing_specs@1`

4.4 Test Program Conversion

4.4.1 CONV CONVMGR

Description	**The "`CONV CONVMGR`" command converts the test program specified by a spreadsheet (in xlsx format) to V93000 test program using TestInsight "`convmgr`" tool. After the conversion is completed, SmartShell adds converted patterns and test suites to the loaded testflow.**
Syntax	`CONV CONVMGR <xlsx_file>`
Parameters	xlsx_file: the spread-sheet file with the path for the convmgr tool to convert the test program
Return Value	Conversion messages from the conversion process
Return Error	Condition: the number of arguments is incorrect Return: ERROR: SYNTAX ERROR: command "`CONV xxx`" correct syntax is Condition: file path does not exist Return: ERROR: <file path> does not exist Condition: There is more than one converted flow in the converted test program Return: ERROR: Path <source path>/output/testflow/ too many flows
Example	>>> CONV CONVMGR /home/useraccount/testfiles/cnvmgr_prod_SKR_A1.xlsx <<< Analyzing testflow __tmp_smartshell_demo.flw... <<< Analyzing pin configuration smartshell_demo.pin... <<< Analyzing timing smartshell_demo.tim.mfh... <<< Running ConvMgr (Version 3.3.3 [linux x86_64_RHEL7] 24-Apr-2020). <<< (c) Copyright 2000-2021 Advantest Corporation <<< [ConvMgr] User Comment : SkyRock-A1_all_scan. <<< [ConvMgr] Target : 93000DD. <<< [ConvMgr] Info : Read spreadsheet. <<< [ConvMgr] Info : Reading done. <<< [ConvMgr] Info : Import spreadsheet. <<< [ConvMgr] Warning : [sheet TestList] Levels file was not defined - ignoring <<< Levels column. <<< [ConvMgr] Info : Import done. <<< [ConvMgr] Info : Converting testlist. <<< [ConvMgr] Warning : Flag 'dd_TestMethod_mode' from conversion setup 'chaca' is invalid - ignored: Name 'utm' is not defined. <<< [ConvMgr] Warning : Flag 'dd_TestMethod_mode' from conversion setup 'setup' is invalid - ignored: Name 'utm' is not defined. <<< [ConvMgr] Info : Conversion done. <<< [ConvMgr] Info : Generating device. <<< [ConvMgr] Info : Generating configuration file. <<< [ConvMgr] Info : Generating timing file. <<< [ConvMgr] Info : Generating levels file. <<< [ConvMgr] Info : Generating pattern files. <<< [ConvMgr] Info : Generating testflow file.

	<<< [ConvMgr] Info : Generating .technology file. <<< [ConvMgr] Info : Generator done. <<< ##### ConvMgr Summary ##### <<< ## Status : Passed <<< ## Up-to-date tests : 4 <<< ## Generated tests : 0 <<< ## Failed tests : 0 <<< ## Device directory : output <<< ## Device top path : output/testflow/SKY_ROCK-A1.tf <<< ## Total conversion time: 0.0 minutes, 15.794224 seconds <<< ADD SUITE: <<< all_dc_ram_tk_ret_pass1_io1_TO_B0_setup <<< all_dc_ram_tk_ret_pass1_io1_TO_B0_scan <<< ADD TM: <<< tm_5 <<< tm_6

4.4.2 CONV UNIX

Description	**The "CONV UNIX" command runs a given UNIX command on the very workstation where test program is being converted and returns the command results back to the calling client.**
Syntax	CONV UNIX=<cmd>
Parameters	cmd: UNIX command
Return Value	Command results of the given UNIX command
Return Error	None
Example	>>>CONV UNIX=cd ../ <<</workspaces/atslxws282

4.5 STIL Pattern Conversion and Registration

The group of commands listed here are specifically designed for application with Siemens EDA software SiliconInsight and ATE-Connect. The STIL pattern conversion happens via `ATEStreamPattern` command from either the user's EDA script. The SmartShell server will automatically register the converted and loaded STIL pattern in together with the created burst label name and created timing set.

Original pin configuration setup may have additional pin groups added to the system for new timing setup. However original pin configuration file remains unchanged. When the user saves the timing setup after the conversion completes, to keep consistent setup across timing setup and pin configuration, the user will need to save the pin configuration in SmarTest Work Center or with SmartShell command:

```
SAVE SETUP PIN
```

Original pattern master file is not changed unitl either manually save it in SmarTest Work Center or with SmartShell command (refer to section 4.1.6)

```
SAVE SETUP VECTOR
```

User may consider to backup the original pattern master file in case unexpected happens.

Original timing master file is not changed instead SmartShell creates new timing master file preceded with `__tmp_` to the original master file name as:

```
__tmp_<original_file>.mfh
```

This file can be used directly or be copied to a new file. The testflow can select this file or a new copied file for further operation.

For execution, the user will simply issue either `ATEExecutePatternGetFailures` or `ATEExecutePatternGoNoGo` with the STIL file name as their argument. Those commands are sent by Siemens SiliconInsight ATEConnect interface.

4.5.1 SAVE LOADED_STIL_PATTERN

Description	**The "`SAVE LOADED_STIL_PATTERN`" command saves the list of STIL patterns which were converted and downloaded to SmarTest via `ATEStreamPattern` command to a csv file. This list is in csv format and each line contains a name of STIL pattern file, correspondent burst label name and timing set for single-port or multi-port timing specification**
Syntax	`SAVE LOADED_STIL_PATTERN <file>.csv`

Parameters	`File.csv:` mapping file located under `test_program_folder/vectors`
	File format:
	`Original STIL File` `Loaded OpSeq Name` `Loaded Spec Set`
	`atpg_scan_chain1.stil` `burst_atpg_scan_chain1` `1,1,1`
	`atpg_scan_chain2.stil` `burst_atpg_scan_chain2` `2,1,1`
	`atpg_scan.stil` `burst_atpg_scan` `mptiming@1,1,1`

Return Value	If no error, pass
Return Error	If no any STIL file loaded with ATEStreamPattern command, it gives a warning:

	WARNING: no any STIL file loaded with ATEStreamPattern command
Example	SAVE LOADED_STIL_PATTERN preloaded_stil_pattern_map.csv

4.5.2 SET LOADED_STIL_PATTERN

Description	**The "SET LOADED_STIL_PATTERN" command sets a list of STIL patterns which were converted and downloaded to SmarTest. This list is in csv format and each line contains a name of STIL pattern file, correspondent burst label name and timing set information. This command supports single-port timing and multi-port timing specification.**
Syntax	SET LOADED_STIL_PATTERN <file>.csv
Parameters	File.csv: mapping file located under test_program_folder/vectors File format: Original STIL File, Loaded OpSeq Name, Loaded Spec Set atpg_scan_chain1.stil, burst_atpg_scan_chain1, 1,1,1 atpg_scan_chain2.stil, burst_atpg_scan_chain2, 2,1,1 atpg_scan.stil, burst_atpg_scan, mptiming@1,1,1
Return Value	If no error, pass
Return Error	If error, showing pattern or timing were not found
Example	SET LOADED_STIL_PATTERN preloaded_stil_pattern_map.csv ERROR: label "burst_atpg_scan_chain1" were not found ERROR: timing setup 2,1,1 was not available

4.5.3 GET LOADED_STIL_PATTERN

Description	**The "GET LOADED_STIL_PATTERN" command returns a list of preloaded STIL patterns which were converted and downloaded to SmarTest.**
Syntax	GET LOADED_STIL_PATTERN
Parameters	None
Return Value	A list of preloaded stil pattern to its correspondent burst label and timing set stil_file_name burst_label_name timin_set stil_file_name burst_label_name timin_set ...
Return Error	If no any preloaded stil pattern, then return WARNING: no preloaded pattern
Example	GET LOADED_STIL_PATTERN atpg_scan_chain1.stil burst_atpg_scan_chain1 1,1,1 atpg_scan_chain2.stil burst_atpg_scan_chain2 2,1,1 atpg_scan.stil burst_atpg_scan mptiming@1,1,1

4.6 Pattern Modification

4.6.1 EDIT VEC – Structure Change

Description	**The "EDIT VEC" command changes the vector data at a given vector number in a given main label**
Syntax	`EDIT VEC <label> <spos>@[[R]<n>@]<data>`
Parameters	`label:` main label (pattern name) for vector data editing `spos:` the vector number `R<n>:` repeat counts and R key can be omitted `data:` state char and x-mode separated by "," and there is only one pin, then the vector data must end with "," too"
Return Value	Update data operation information , pass
Return Error	Condition: the number of arguments is incorrect Return: ERROR: SYNTAX_ERROR: - command : DFT_EDIT_VEC, argument :xxx Condition: the provided vector number is not a number Return: ERROR: the provided vector number xxx is not a number Condition: the provided vector number is out of range Return: ERROR: the provided vector number "-1" is not a number Condition: the pattern name that you input is not exist Return: ERROR: Pattern sLabel does not exist Condition: the pattern is a Multiport burst Return: ERROR: Pattern xxx is a Multiport burst. Condition: the pattern is a Main Label Return: ERROR: Pattern xxx is a Main Label. Condition: the provided state has no mapping in timing Return: ERROR: can not find mapping pf state xxx Condition: the number of pins is not equal to the number of provided data Return: ERROR: The number of pins is not equal to the number of data.

Example	>>>EDIT VEC demo_jtag1 2@00H00
	<<<pattern demo_jtag1 with value 00H00
	<<<The mapped value is 0,0,1,0,0,
	<<<pass
	
	example x5-mode for 6 pins vector pattern:
	`>>> EDIT VEC pat1`
	`5@R2@10000,LXHLL,10010,11111,LHLHL,00000`
	`<<<pass`
	where it starts editing from position 5 and repeats 2 with data 100000
	
	example x5-mode for 6 pins vector pattern:
	`>>>EDIT VEC pat1 5@10000,LXHLL,10010,11111,LHLHL,00000`
	`<<<pass`
	
	example x5-mode for 1 pins vector pattern:
	`>>>EDIT VEC pat1 5@10000,`
	`<<<pass`
	where it starts editing from position 5 with data 100000

	example x1-mode for 6 pins vector pattern:
	`>>>EDIT VEC pat1 5@101010`
	`<<<pass`

4.6.2 EDIT VEC – Structure Change

Description	**The "`EDIT VEC`" command changes the pattern structure at a given vector number in a given main label**
Syntax	`EDIT VEC <label> <vpos>@R<#>@#<#>`
Parameters	`label:` main label (pattern name) for vector data editing `vpos:` the vector number `R<n>:` repeat counts and R key can be omitted `#<n>:` n vectors to be repeated It creates a loop for several vectors; For an existing loop structure that matches the instruction, only loop count is modified.
Return Value	Update data operation information , pass
Return Error	Any existing loop structure that doesn't match what the instruction looks for results in an error.

Example	`>>>EDIT VEC demo_jtag1 2@R3@#3`
	`<<<pass`
	Starting at vector number 2, creates a loop vector structure with 3 for the 3 vectors in it.

4.6.3 INSERT VEC

Description	**The "`INSERT VEC`" command inserts the vector data at a given vector number in a given label and optionally with a repeat count**
Syntax	`INSERT VEC <label> <ipos>@R<n>@data` `INSERT VEC <label> <ipos>@data`
Parameters	`label:` main label for vector data editing `spos:` the vector number `R<n>:` repeat counts and R key can be omitted `data:` state char and x-mode separated by "," and only vector it must end with "," too"
Return Value	Pass
Return Error	Condition: the number of arguments is not right Return: ERROR: Syntax error – command : DFT_EDIT_VEC, argument : xxxx Condition: the provided vector number is not a number Return: ERROR: the provided vector number xxx is not a number. Condition: the provided vector number is out of range Return: ERROR: the provided vector number xxx is out of range; the last vector number is xxx Condition: the provided state has no mapping in timing Return: ERROR: can not find mapping pf state xxx

	Condition: the number of pins is not equal to the number of provided data Return: ERROR: The number of pins is not equal to the number of data

Example	`>>>INSERT VEC pat1 5@R5@100`
	`<<<pass`
	`>>>INSERT VEC pat1 5@100`
	`<<<pass`

4.6.4 DELETE VEC

Description	**The "`DELETE VEC`" deletes vector at a given vector line number for a given main label**
Syntax	`DELETE VEC <label> <ipos>`
Parameters	`label`: main label for vector data editing `ipos`: the vector number
Return Value	Pass
Return Error	Condition: the number of arguments is not right Return: ERROR: Syntax error – command : DFT_EDIT_VEC, argument : xxxx Condition: the provided vector number is not a number Return: ERROR: the provided vector number xxx is not a number. Condition: the provided vector number is out of range Return: ERROR: the provided vector number xxx is out of range; the last vector number is xxx
Example	`>>>DELETE VEC jtag_patter 5` `<<<pass`

4.6.5 EDIT VEC SIGNAL_LIST

Description	**The "`EDIT VEC SIGNAL_LIST`" command is a pin-based vector editing command to edit vector at a given vector line number**
Syntax	`EDIT VEC <label> <pin>+ <spos>@<data>&+`
Parameters	`label` : main label for vector data editing `<pin>,+`: list of pin separated by comma “,” `spos` : the start vector number `epos` : the end vector number `data` : state char and x-mode separated by “,” and only one vector it must end with “,” too"
Return Value	pass
Return Error	Condition: the number of arguments is not right Return: ERROR]:Syntax error – command : DFT_EDIT_VEC, argument : xxxx Condition: the provided vector number is not a number Return: ERROR: the provided vector number xxx is not a number. Condition: the provided vector number is out of range Return: ERROR: the provided vector number xxx is out of range; the last vector number is xxx Condition: the provided state has no mapping in timing Return: ERROR: can not find mapping pf state xxx

	Condition: the number of pins is not equal to the number of provided data Return: ERROR: The number of pins is not equal to the number of data.
Example	example x5-mode: >>>EDIT VEC label1 pin1 5@XXXXX&XXXXX <<<pass >>>EDIT VEC label1 pin1,pin2 5@XXXXX, <<<pass example x1-mode: >>>EDIT VEC label1 pin1 5@X&X <<<pass >>>EDIT VEC label1 pin1 5@X <<<pass

4.6.6 SPLIT OPSEQ

Description	**The "SPLIT OPSEQ" command extracts the portion of an existing source pattern from a start vector number to a stop vector number**
Syntax	SPLIT OPSEQ <label> SRC=<srclabel> PRE=<prefix> PORT=<port> STARTC=<spos> STOPC=<epos>
Parameters	label : main label for vector data extraction srclabel: source multiport burst label prefix : adding a prefix to the main label port : port under which the split begins spos : start vector number epos : stop vector number
Return Value	list of created pattern labels;
Return Error	ERROR: Syntax error, port name must be defined ERROR: Start vector number is not specified, make 0 as the default value. ERROR: Stop vector number is not specified, make the end as the default value.
Example	>>>SPLIT OPSEQ new_label SRC=old_pattern PORT=jtag_port STARTC=10 STOPC=15 the ecyc end cycle is across multiple label, the new label is listed as: <label_scyc>_max1 <label>_(max1+1)_(max2) ... <label>_(pre_max+1)_(maxn) <label>_(maxn+1)_(ecye)\n SPLIT OPSEQ SRC=old_pattern PORT=jtag_port STARTC=10 STOPC=15 the ecyc end cycle is across multiple label, the new label is listed as:

	<main_label_name_scyc>_max1 <next_main_label_name>_(max1+1)_(max2) ... <next_main_label_name>_(pre_max+1)_(maxn) <next_main_label_name>_(maxn+1)_(ecye)\n SPLIT OPSEQ SRC=old_pattern PRE=new_label PORT=jtag_port STARTC=10 STOPC=15 new_label_<main_label_name_scyc>_max1 new_label_<next_main_label_name>_(max1+1)_(max2) ... new_label_<next_main_label_name>_(pre_max+1)_(maxn) new_label_<next_main_label_name>_(maxn+1)_(ecye)"

4.7 Spec and Flow Variable Value

4.7.1 SET FLOW VAR

Description	**The "SET FLOW VAR" command sets a value to a flow variable**
Syntax	SET FLOW [<flow>] VAR INT/DOUBLE/STRING "<var>=<value>
Parameters	flow : name of the test flow var : variable name value: value"
Return Value	Pass
Return Error	None
Example	>>>SET FLOW VAR INT var1=0.5 <<<pass

4.7.2 SET SPEC VAR

Description	**The "SET SPEC VAR" command sets a value to a spec variable defined in the primary specification or a given specification**
Syntax	SET SPEC [<specname>] VAR TIM/LEV <var>=<value>
Parameters	specname : spec name optional var : variable name value : value
Return Value	pass no error
Return Error	**Condition:** The spec does not exist. **Return:** ERROR: Level/Timing spec xxx does not exist!
Example	>>>SET SPEC VAR LEV Vcoef=0.5 <<<pass

4.8 Level and Edge Settings

4.8.1 SET LEV

Description	**The "SET LEV" command sets level values to level resources such as vol, voh, vil, vih, etc.**
Syntax	SET LEV <pin>,+ <levelkeyword>=<value>+
Parameters	pin : pin name or list of pins separated by comma “,” levelkeyword: such vih, vil, vol, vol, etc. value : value in mV"
Return Value	Pass
Return Error	WARNING: Invalid level param xxx! WARNING: The mode of pin xxx is OFF, can not be reset!
Example	>>>SET LEV pin,pin2 vih=1000 vil=300 <<<pass

4.8.2 SET LEV_DPS

Description	**The "SET LEV_DPS" command sets a force voltage value to a DPS pin**
Syntax	SET LEV_DPS <pin>=<value>
Parameters	pin: DPS pin name value: value in mV
Return Value	Pass
Return Error	**Condition:** the specified pin does not exist **Return:** ERROR: Pin xxx is not A DPS pin
Example	>>>SET LEV_DPS VDD 0.9 <<<pass

4.8.3 SET EDGE

Description	**The "SET EDGE" command sets timing edge values to timing edges**
Syntax	SET EDGE <pin>,+ (<edge>=<value>+) The specification of position string needs be encapsulated with ( )
Parameters	pin : pin name or list of pins separated by comma”,” edge : d#, r# or e# value : value in ns"
Return Value	pass no error
Return Error	Condition: the specified pin is not defined in the original timing waveform Return: ERROR: xxx was not defined in the original timing waveform!
Example	>>>SET EDGE pin1 (d1=1 d2=2.5) <<<pass >>>SET EDGE pin1,pin2 (d1=1 d2=2.5) <<<pass

4.9 TestMethod Control

4.9.1 SET SUITE TM_LINK

Description	**The "SET SUITE TM_LINK" command creates a temporary test suite for a given TestMethod, specs and opseq**
Syntax	SET SUITE [<suite_name>] TM_LINK=<tm> SPEC=<levelset>&<timingset> OPSEQ=<label> SET SUITE [<suite_name>] TM_LINK=<tm> SPEC=<specs> OPSEQ=<opseq>
Parameters	SET SUITE [<suite_name>] TM_LINK=mytm SPEC=<levelset>&<timingset> OPSEQ=<label> suite_name: If this is not provided, internally a tmpsuite is created mytm : TestMethod name such tml.DC.General_PMU levelset : <eqnset>,<specname>,<setno> timingset : <eqnset>,<specname>,<setno> or specname@<setno>,+ label : either multi-port burst label or single-port burst label or main label depends on timing specs
Return Value	Pass
Return Error	Condition: the TestMethod can not be found. Return: ERROR: No valid Test method specified. Condition: the given pattern does not exist. ERROR: Pattern xxx does not exist.
Example	>>>SET SUITE TM_LINK=mytm SPEC=99,1,1&specjtab@1,1,1 OPSEQ=label <<<pass This uses the parameter as primary setups for test suite >>>SET SUITE TM_LINK=mytm SPEC=99,1,1&() OPSEQ=() <<<pass This clears the existing timing and label to null >>>SET SUITE TM_LINK=mytm SPEC=99,1,1& <<<pass This uses existing timing and label as primary setup. setup parameters are ignored, then the execution relies on TestMethod for specification and operating sequence

4.9.2 SET SUITE TM_VALUE

Description	**The "SET SUITE TM_VALUE" command assigns values to TestMethod parameters for the temporary test suite or a given test suite.**
Syntax	SET SUITE [<suite_name>] TM_VALUE <"parameter=value"> +
Parameters	Quote around "parameter=value" are needed. suite_name: If this is not provided, it uses internally created tmpsuite which was created by SET SUITE TM_LINK parameter : TestMethod parameter name value : value to be assigned to"
Return Value	pass

Return Error	Condition: the parameter does not exist and so on Return: ERROR: can not be set up the parameter properly
Example	>>>SET SUITE TM_VALUE "var1=0.3"8 "var2=0.8 <<<pass

4.9.3 GET SUITE TM_PARAM

Description	**The "GET SUITE TM_PARAM" command lists all TestMethod parameter names in the temporary test suite created by the SmartShell "SET SUITE" command or a given suite <testsuite> in the current testflow.**
Syntax	GET SUITE [<suite_name>] TM_PARAM
Parameters	suite_name: test suite name
Return Value	var1 var2
Return Error	Condition: the suite name is not defined in test flow. Return: ERROR: The suite name may not be defined in testflow. Condition: the parameter does not exist and so on Return: ERROR: can not query the parameter properly
Example	*From the temporary test suite:* >>>GET SUITE TM_PARAM *From an existing test suite:* >>>GET SUITE pll_test TM_PARAM

4.9.4 GET SUITE TM_PARAM_WITH_VALUE

Description	**The "GET SUITE TM_PARAM_WITH_VALUE" command lists all TestMethod parameter names and their values in the temporary test suite created by the "SET SUITE" command or a given test suite <testsuite>**
Syntax	GET SUITE [<suite_name>] TM_PARAM
Parameters	suite_name: test suite name
Return Value	var1=value1 var2=value2
Return Error	Condition: the test suite name is not defined in test flow. Return: ERROR: The test suite name may not be defined in testflow. Condition: the parameter does not exist and so on Return: ERROR: can not query the parameter properly
Example	*From the temporary test suite:* >>>GET SUITE TM_PARAM *From an existing test suite:* >>>GET SUITE pll_test TM_PARAM

4.9.5 GET SUITE TM_VALUE

Description	**The "GET SUITE TM_VALUE" command returns the value of a TestMethod parameter in the temporary test suite created by the SmartShell "SET SUITE" command or a given test suite <testsuite>**
Syntax	GET SUITE [<test suite>] TM_VALUE "<var>" +

Parameters	`testsuite` : test suite name `var` : TestMethod parameter name `Notes` : need quote " surrounding each variable name
Return Value	`<var>=value/parameter`
Return Error	None

Example	*From the temporary test suite:*
	`>>>GET SUITE TM_VALUE "settlingtime"`
	From an existing test suite:
	`>>>GET SUITE pll_test TM_VALUE "settlingtime" "output"`

4.10 Suite Flag Control

4.10.1 GET SUITE FLAG

Description	**The "GET SUITE FLAG" command lists all the suites flag names and their values of the given test suite**
Syntax	`GET SUITE <suite_name> FLAG [BYPASS]`
Parameters	`suite_name:` The name of the test suite.
Return Value	`BYPASS=1`
Return Error	Condition: The test suite name is not defined in testflow. Return: ERROR: The test suite name may not be defined in the testflow.

Example	*From the existing test suite:*
	`>>>GET SUITE suite1 FLAG BYPASS`
	`<<<BYPASS=1`

4.10.2 SET SUITE FLAG

Description	**The "SET SUITE FLAG" command sets the flag BYPASS of a given test suite**
Syntax	`SET SUITE <suite_name> FLAG BYPASS <value>`
Parameters	`suite_name:` The name of the test suite. `value`: 1 or 0
Return Value	`pass`
Return Error	Condition: The test suite name is not defined in the testflow. Return: ERROR: The test suite name may not be defined in the testflow.

Example	*From the existing test suite:*
	`>>>SET SUITE suite1 FLAG BYPASS 1`
	`<<<pass`

4.11 Add and Remove Setups

4.11.1 ADD PAT

Description	**The "ADD PAT" command adds new <pattern file> to the existing pattern list in SmarTest.**
Syntax	ADD PAT <newpatternlistfile>
Parameters	newpatternfile: pattern with path to be added
Return Value	pass no error
Return Error	ERROR: Fail to load binary vector file from pat_path" ERROR: File xxx does not exist" ERROR: can not open pattern file xxx" ERROR: The same pattern file exist"
Example	>>>ADD OPSEQ /home/demo/vectors/newlabel.binl <<<pass >>>ADD OPSEQ /home/demo/vectors/newlabel <<<pass

4.11.2 ADD OPSEQ

Description	**The "ADD OPSEQ" command adds new <burst file> to the existing pattern list in SmarTest.**
Syntax	ADD OPSEQ <newburstfile>
Parameters	newburstfile: burst with path to be added
Return Value	pass no error
Return Error	ERROR: Fail to load binary vector file from pat_path" ERROR: File xxx does not exist" ERROR: can not open pattern file xxx" ERROR: The same pattern file exist"
Example	>>>ADD OPSEQ /home/demo/vectors/newburst <<<pass

4.11.3 ADD PMF

Description	**The "ADD PMF" command adds new <pattern master file> to the existing pattern list in SmarTest.**
Syntax	ADD PMF <newpatternmasterfile>
Parameters	newpatternmasterfile: pattern master file with path to be added
Return Value	pass no error
Return Error	ERROR: Fail to load binary vector file from pat_path" ERROR: File xxx does not exist" ERROR: can not open pattern file xxx" ERROR: The same pattern file exist"
Example	>>>ADD OPSEQ /home/demo/vectors/newlabel.pmfl <<<pass

4.11.4 ADD SETUP

Description	**The "ADD SETUP" command adds timing setup to the existing master file**
Syntax	ADD SETUP <specsfiles>
Parameters	specsfiles: specification file to be added to the master file
Return Value	DFT_TIM_APPEND Finished
Return Error	None
Example	>>>ADD SETUP /home/demo/timing/subfold/timing1.setup <<<pass

4.11.5 REMOVE OPSEQ

Description	**The "REMOVE OPSEQ" command removes <label> from the pattern list in the test program, the file itself remains untouched.**
Syntax	REMOVE OPSEQ <label>
Parameters	label: multi-port burst label or single-port burst label or main label
Return Value	pass no error
Return Error	Condition: the label does not exist Return: ERROR: can not find label xxx at any port. Please double check.
Example	>>>REMOVE OPSEQ jtag_temp_pattern <<<pass

4.12 Test Setup Query

4.12.1 GET BOARD_CONF

Description	**The "GET BOARD_CONF" command returns pin configuration**
Syntax	GET BOARD_CONF PIN/PORT/GROUP
Parameters	PIN : list all pins PORT : list all ports GROUP: list all groups
Return Value	Complete list of signals, ports or pin groups
Return Error	None
Example	>>>GET BOARD_CONF PIN <<<"BADD0 I BADD1 I BADD2 I …… >>>GET BOARD_CONF PORT <<<"EXAMPLE1 EXAMPLE2 EXAMPLE3……" …… >>>GET BOARD_CONF GROUP <<<"GROUP1 GROUP2 GROUP3……"

4.12.2 GET SIGNAL_LIST GROUP

Description	**The "GET SIGNAL_LIST GROUP" command returns the complete signal list from the given group**
Syntax	GET SIGNAL_LIST GROUP <group>
Parameters	group : name of pin group
Return Value	Signal list
Return Error	Condition: the group does not exist Return: ERROR Group name xxx is invalid!
Example	>>>GET SIGNAL_LIST GROUP test_pin_group <<<"PIN1 PIN2 PIN3 PIN4 PIN5 PIN6……"

4.12.3 GET SIGNAL_LIST PORT

Description	**The "GET SIGNAL_LIST PORT" command returns the complete signal list from the <port>**
Syntax	GET SIGNAL_LIST PORT <port>
Parameters	group: name of pin group
Return Value	Signal list
Return Error	Condition: the port does not exist Return: ERROR: Group name xxx is invalid!
Example	>>>GET SIGNAL_LIST PORT TEST_PORT <<<"PIN1 PIN2 PIN3 PIN4 PIN5……"

4.12.4 GET PAT PORT_NAME

Description	**The "GET PAT PORT_NAME" command returns the port name from a given pin in a given multi-port label**
Syntax	`GET PAT <label> PORT_NAME <pin>`
Parameters	`label:` multi-port burst label `pin :` a pin
Return Value	Port name of the specified pin
Return Error	Condition: the label does not exist Return: ERROR: Pattern xxx does not exist.
Example	`>>>GET PAT demo_pat_group PORT_NAME pin1` `<<<PORT1`

4.12.5 GET PAT SIGNAL_LIST

Description	**The "GET PAT SIGNAL_LIST" command lists all signals in a given <label>**
Syntax	`GET PAT <label> SIGNAL_LIST`
Parameters	`label:` main label
Return Value	Signal list
Return Error	Condition: the label does not exist Return: ERROR: Pattern xxx does not exist. Condition: the label is a multiport burst Return: Error Pattern xxx is a Multiport burst.
Example	`>>>GET PAT demo_pat_group SIGNAL_LIST` `<<<PIN1 PIN2 PIN3 PIN4 PIN5……`

4.12.6 GET PAT LABEL_LIST MULTI_PORT

Description	**The "GET PAT LABEL_LIST MULTI_PORT" command lists all multi-port burst labels and associated ports**
Syntax	`GET PAT LABEL_LIST MULTI_PORT`
Parameters	None
Return Value	Label list
Return Error	None
Example	`>>>GET PAT LABEL_LIST MULTI_PORT` `<<<LABEL1 LABEL2 LABEL3`

4.12.7 GET PAT LABEL_LIST SINGLE_PORT

Description	**The "GET PAT LABEL_LIST SINGLE_PORT" command lists all single-port (@) burst labels**
Syntax	`GET PAT LABEL_LIST SINGLE_PORT`
Parameters	None
Return Value	Label list
Return Error	WARNING: no results is found

Example	>>>GET PAT LABEL_LIST SINGLE_PORT <<<example_pat

4.12.8 GET PAT LABEL_LIST

Description	**The "GET PAT LABEL_LIST" command lists all MAIN pattern names for single port burst label or for multi-port burst label of a given main port**
Syntax	GET PAT <label> LABEL_LIST <port>
Parameters	label: multi-port burst label name port : port name
Return Value	List of main pattern labels
Return Error	**Condition:** the burst label does not exist **Return:** ERROR: Pattern xxx does not exist **Condition:** the port does not exist. **Return:** ERROR: Port xxx might not exist
Example	>>>GET PAT multi_port_burst_label LABEL_LIST demo_port <<<pat1 demo_port <<<pat2 demo_port

4.12.9 GET PAT PRIMARY

Description	**The "GET PAT PRIMARY" command returns the primary label name**
Syntax	GET PAT PRIMARY
Parameters	None
Return Value	Active label name
Return Error	None
Example	>>>GET PAT PRIMARY <<<primary_label1

4.12.10 GET PAT VEC

Description	**The "GET PAT VEC" command gets vector data at a given vector number**
Syntax	GET PAT <label> VEC <vec_number>
Parameters	label: main label vec_number: the vector number
Return Value	<data>[,+] where <data> is vector data separated by "," and "&". In x1 mode, "&" is omitted and no blank in between. *Example:* 00000 which can either be 1 x5- or 5 x1-vector data. Issue command GET XMODE to figure it out. *Example:* 00000,00000 which shows 1 x5-vector data of 2 pin data.
Return Error	**Condition:** the label doesn't exist

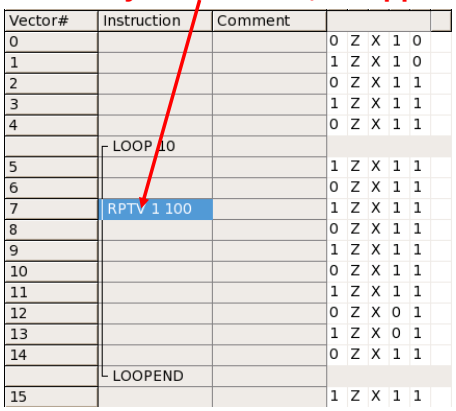
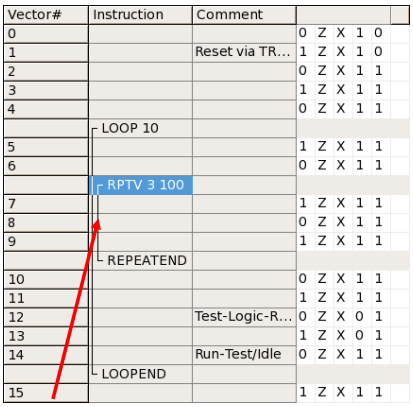
	Return: Error: Pattern xxx does not exist. **Condition:** the vector number exceeds the limit. **Return:** Error: the provided vector number xxxx is out of range, the last vector number is <max number>.
Example	>>>GET PAT demo_pat VEC 5 <<<00L00 1

4.12.11 GET PAT VEC by SIGNAL

Description	**The "GET PAT VEC by SIGNAL" command gets vector data from start vector to stop vector for a given signal**
Syntax	GET PAT <label> VEC <pin> <start> <stop>
Parameters	label: main label name pin : pin start: start vector number stop : stop vector number"
Return Value	<data>[,+[&]] where <data> is vector data separated by "," and "&". In x1 mode, "&" is omitted and no blank in between. *Example:* 00000 which can either be 1 x5- or 5 x1-vector data. Issue command GET XMODE to figure it out. *Example:* 00000&00000&10101 which shows 3 x5-vector data of 1 pin data. pin does not belong to the given main pattern pin list, error: ERROR: "<pin>" doesn't belong to the given main pattern label "<label>" <stop> is outside of max available vector range, error: ERROR: "<stop>" is outside of max available vector range the physical waveform index PW <number> from pattern <label> is not found in the wave table, error: ERROR: "<name of the wave table>" doesn't contain the mapping for waveform index "<number>" given by pattern label "<label>" ***NOTES*** 1) pattern port and wave port can be different; 2) pin in pattern port can be in a timing port with different port name
Return Error	ERROR: "the label <label> doesn't contain the provided pin <pin> under the port <xxxxxx> ERROR: "<stop>" vector number <xxxx> is outside of max available vector range, the last vector number is <max_number> ERROR: "<name of the wave table>" doesn't contain the mapping for waveform index "<number>" given by pattern label "<label>"

Example	`>>>GET PAT demo_pat_group VEC pin1 5 10` `<<<LLLLLL`

4.12.12 GET PAT VNO CYC

Description	**The "`GET PAT VNO CYC`" command gets vector info from the given port and given cycle number. It supports also nested loop which contains one or more single-line repeat(s). It doesn't support nested loop which contains repeat block containing multiple lines of vector again. The following figures show the two pattern cases.** **If the cycle falls here, it supports**   **If the cycle falls here, it does not support**
Syntax	`GET PAT <multiport_label> VNO <port> CYC=<ipos>`
Parameters	multiport_label : multi-port burst label name port : port ipos : cycle number"
Return Value	The vector info of the given cycle
Return Error	Condition: the label is not a multiport burst Return: ERROR: Pattern demo_jtag1 is a Main Label Condition: the label does not exist Return: ERROR: Pattern jtag_demo11 does not exist Condition: the port does not exist Return: ERROR: The specified port xxx does not exist

Example	`>>>GET PAT demo_jtag1 VNO pJTAG CYC=5`
	`<<<demo_jtag1:5 L0 R1 0`
	`main_vector_label_name1`
	`...`
	`main_vector_label_nameX:#vecnum L# R# p n L# R# m`
	where `#vecnum` is the vector number in the main pattern
	`L#:` >1 there are **multiple lines of vector repeat**
	`R#:` total repeat count
	`p :` relative vector number within **multiple lines of vector repeat** block
	`n :` number of loop counts to get to this cycle for **multiple lines of vector repeat**
	`m :` number of repeat counts to get this cycle in a **single line of vector repeat**
	Example:

	label_1_X5 label_2_X5:10665 L0 R1 0 *Explanation:* the given cycle number is associated with main label “label_2_X5” L0 means no multiple lines of vectors for the repeat R1 means one single vector 0 means it is at the 0 position of the vector line *Example:* >>> GET PAT mpb_label1 VNO port1 CYC=11150 <<< label_1_Y7 label_2_Y5:10 L10 R14 4 10 L0 R119 20 *Explanation:* the given cycle number is located at main label “label_2_Y5” L10 means **10-line vectors** repeat R14 means the **10-line vectors** repeats for 14 times 4 means the given cycle is at 4th vector position in this **10-line vectors** 10 means the **10-line vectors** at the 10th loops L0 means there is a single line vector repeat R119 means this line of vector repeats for 119 20 means this cycle is at 20th repeat count So, there could be of total 119+9=128 cycles within this repeat block there is only one single-line repeat. To figure it out exactly how many single-line repeat, we can use GET PAT VEC command to figure it out

4.12.13 GET PAT TOTAL_CYC

Description	**The "GET PAT TOTAL_CYC" command gets total cycle number in a given main label**
Syntax	GET PAT <label> TOTAL_CYC
Parameters	label : main label name
Return Value	total cycle number
Return Error	Condition: the label does not exist Return: ERROR: Pattern xxx does not exist Condition: the label is not a multiport burst Return: ERROR: Pattern xxx is a Multiport burst
Example	>>>GET PAT demo_jtag1 TOTAL_CYC <<<32

4.12.14 GET PAT TOTAL_VEC

Description	**The "GET PAT TOTAL_VEC" command returns the total vector number in a given main label.**
Syntax	GET PAT <label> TOTAL_VEC
Parameters	label: main label name
Return Value	Condition: the label does not exist Return: ERROR: Pattern xxx does not exist

	Condition: the label is not a multiport burst Return: ERROR: Pattern xxx is a Multiport burst
Return Error	None
Example	>>>GET PAT demo_jtag1 TOTAL_VEC <<<32

4.12.15 GET FLOW SUITE_LIST

Description	**The "GET FLOW SUITE_LIST" command lists all test suite names in a current working flow or a given flow.**
Syntax	GET FLOW SUITE_LIST
Parameters	flow: flow name
Return Value	list of all test suites under this testflow <flow>
Return Error	None
Example	>>>GET FLOW SUITE_LIST <<<test1 <<<test2 <<<jtag_demo

4.12.16 GET FLOW VAR_LIST

Description	**The "GET FLOW VAR_LIST" command lists all variables in the current working flow**
Syntax	GET FLOW VAR_LIST TYPE
Parameters	TYPE: one of STRING/DOUBLE/INT
Return Value	list of all variables
Return Error	None
Example	>>>GET FLOW VAR_LIST INT <<<"var1 <<<var2 <<<"

4.12.17 GET FLOW VAR

Description	**The "GET FLOW VAR" command returns the variable value in the current working flow**
Syntax	GET FLOW VAR TYPE <var>
Parameters	TYPE: one of STRING/DOUBLE/INT var : variable name"
Return Value	value of the testflow variable
Return Error	None
Example	>>>GET FLOW VAR STRING var1 <<< var1 "hello smartshell"

4.12.18 GET LEV

Description	**The "GET LEV" command obtains the level setup information for a given <pin>**
Syntax	`GET LEV <pin>`
Parameters	`pin:` pin name
Return Value	`Pin=xxx vil=0 vih=1.8 vol=0.5 voh=0.7 iol=0.1 ioh=0.2……`
Return Error	None
Example	`>>>GET LEV test_pin1` `<<<pin=test_pin1 vil=0 vih=1.8 vol=0.5 voh=0.7 vcl=-2 vch=7 iol=0.1 ioh=0.2`

4.12.19 GET LEV_DPS

Description	**The "GET LEV_DPS" command returns the force value of DPS pin**
Syntax	`GET LEV_DPS [<pin>]`
Parameters	`pin:` pin name
Return Value	`Pin value`
Return Error	Condition: pin is not a DPS pin Return: ERROR: Pin xxx is not A DPS pin Condition: pin doesn't exist Return: ERROR: Pin xxx does not exist
Example	`>>>GET LEV_DPS VDD` `<<<VDD 0.8`

4.12.20 GET EDGE

Description	**The "GET EDGE" command gets timing edge setup for a given pin**
Syntax	`GET EDGE <pin>`
Parameters	`pin:` pin name
Return Value	`pin_name d1:0 d2:1`
Return Error	Condition: no edge information is stored Return: WARNING: no results is found
Example	`>>>GET EDGE TDO` `<<<TDO d1:0 r1:14`

4.12.21 GET SPEC VAR_LIST

Description	**The "GET SPEC VAR_LIST" command lists all spec variable names in the current specification or a given specification**
Syntax	`GET SPEC [<specification/eqn_no,spec_no>] VAR_LIST [TIM/LEV]`
Parameters	specification: specification name for multi-port eqn_no,spec_no: equation number and spec number
Return Value	all spec variables of given type
Return Error	**`Condition:`** `no primary level/timing used`

	Return: ERROR: No level used **Return:** ERROR: No timing used
Example	>>>GET SPEC VAR_LIST TIM <<<E9 Frequency_1 >>>GET SPEC VAR_LIST LEV <<<spec_var1 spec_var2 spec_var3 ……

4.12.22 GET SPEC VAR

Description	**The "GET SPEC VAR" command returns the spec variable value in the current specification or a given specification.**
Syntax	GET SPEC [<specification/eqn_no,spec_no>] VAR <type> <var>
Parameters	type: TIM/LEV Var: the variable specification: specification name for multi-port eqn_no,spec_no: equation number and spec number
Return Value	value of the spec variable
Return Error	Condition: <spec_var> is not found. Result: ERROR: occurred in SVLR? LEV, PRM, ,"spec_var
Example	>>>GET SPEC VAR LEV spec_var <<<DFT_QUERY_ATE_INFO SPEC VAR LEV spec_var "2.5"

4.12.23 GET SPEC LEV EQN

Description	**The "GET SPEC LEV [EQN]" command lists either all the defined level sets or the current active one or the one which is specified by <eqn_number> or <eqn_name>**
Syntax	GET SPEC [<eqn_number/eqn_name>/ALL/ACTIVE] LEV [EQN]
Parameters	eqn_number : equation number
Return Value	EQN is given, it returns equation If… EQNSET 5: "DC_TEST" LEVSET 3: "DC_Strict_in_Relax_out" SPECSET 2: level_spec_low
Return Error	None
Example	>>>GET SPEC ALL LEV EQN EQNSET 1: "Digital" LEVSET 1: "No Active Termination" SPECSET 1: level_spec1 >>>GET SPEC 1 LEV EQN <<<EQNSET 1: "Digital" SPECS

	spec_var1 spec_var2 …… EQUATIONS LEVELSET 1 "No Active Termination" PINS pin1 pin2 pin3 vil = 0 vih = vdd_0 vol = vdd_0 / 2.0 – 0.05 voh = vdd_0 / 2.0 + 0.05 …… >>>GET SPEC ACTIVE LEV EQN <<<EQNSET 1: "Digital" LEVSET 1: "No Active Termination" SPECSET 2: level_spec_low

4.12.24 GET SPEC TIM EQN

Description	**The "GET SPEC TIM EQN" command retrieves the complete equation definition setup aka EQNSET for the primary timing setting or for a given name or a given equation set number or for all single port timing equation sets if any**
Syntax	GET SPEC [<eqn_number/eqn_name>/ALL/ACTIVE] TIM EQN
Parameters	None
Return Value	EQNSET 1: "PLL_TEST" TIMINGSET 1: "PLL_out_test"
Return Error	None
Example	>>>GET SPEC ALL TIM EQN <<< EQNSET 12: "Tim_eqn_clk_EQNSET" TIMSET 1: "tset2" SPECSET 1: Tim_Spec1 EQNSET 13: "Tim_eqn_static_EQNSET" TIMSET 1: "tset2" SPECSET 1: Tim_Spec1 EQNSET 14: "Tim_eqn_reg_X5_EQNSET" TIMSET 1: "tset2" SPECSET 1: Tim_Spec1 …… >>>GET SPEC ALL TIM <<<"timing_spec_3_ports_short" "timing_spec_3_ports_full" "timing_spec_3_ports_part" >>>GET SPEC ACTIVE TIM EQN <<<timing_spec_3_ports_part <<<PORT: GROUP_CLK EQNSET 12: "Tim_eqn_clk_EQNSET" TIMSET 1: "tset2" SPECSET "timing_spec_3_ports_part" <<<PORT: GROUP_REST EQNSET 13: "Tim_eqn_static_EQNSET" TIMSET 1: "tset2" SPECSET "timing_spec_3_ports_part" <<<PORT: GROUP_MAIN EQNSET 14: "Tim_eqn_reg_X5_EQNSET" TIMSET 1: "tset2" SPECSET "timing_spec_3_ports_part"

4.12.25 GET SPEC TIM

Description	**The "GET SPEC TIM" command gets the primary multi-port timing specification or all available multi-port timing specifications**
Syntax	GET SPEC [ALL/ACTIVE] TIM
Parameters	None
Return Value	Specification w/ [port: pinlist]
Return Error	None
Example	>>>GET SPEC ACTIVE TIM <<<spec1 port1: pin1, pin2, pin3, pin4, pin5 …… >>>GET SPEC ALL TIM <<<spec1 spec2 spec3

4.12.26 GET SPEC PERIOD

Description	**The "GET SPEC PERIOD" command returns the test period in the primary timing specification or a given specification for a given port or a given pin name**
Syntax	GET SPEC [<specification>] PERIOD <port_name/pin_name>
Parameters	port_name: port name pin_name : pin name
Return Value	test period for the given port or pin
Return Error	None
Example	>>>GET SPEC PERIOD <port1/pin1> DFT_QUERY_ATE_INFO PORT_PERIOD port1/pin1 40

4.12.27 GET XMODE

Description	**The "GET XMODE" command returns the x mode for a given pin in the current primary timing specification or a given timing specification**
Syntax	GET XMODE <port_name>/<pin_name>
Parameters	pin_name: pin name port name: port name
Return Value	An integer number
Return Error	Condition: the pin_name does not exist Return: ERROR: Pin does not exist
Example	>>>GET XMODE <port_name>/<pin_name> <<<1

4.12.28 GET DVCMAP

Description	**The " GET DVCMAP" command gets the DVC mapping info**

Syntax	`GET DVCMAP <pin_name>`
Parameters	pin_name: the pin name in Waveform definition
Return Value	Waveform definition `<pin> <device_cycle_name> <physical_waveform_index> <definition_string>`
Return Error	**Condition**: the specified pin name does not exist in Waveform definition **Return**: WARNING: no results is found

Example	`>>>GET DVCMAP pin1`
	`<<<pin1 0 0 {d1:!Z d2:F0N}`
	`pin1 1 1 {d1:!Z d2:F1N}`

4.12.29 GET WAVETBL

Description	**The "`GET WAVETBL`" command returns the mapping wavetable name of a given port in use.**
Syntax	`GET WAVETBL <port>`
Parameters	port: the port name
Return Value	The relevant wavetable name of the given port
Return Error	**Condition**: port name does not exist **Return**: WARNING: no results is found
Example	`>>>GET WAVETBL port1` `<<<Wvt_JTAG`

4.13 Execution Results Query

4.13.1 GET FAILC

Description	**The "GET FAILC" command gets failing cycle information with start cycle and no of fails. Before using this command, "EXE" command must be executed. Details see "EXE" command**
Syntax	GET [FAILC] [FAILC_CNT] [FAILC_PIN] [<pinlist>] [STARTC=<spos>] [NO_OF_CYCLES=n>]
Parameters	pinlist: pin list from which the result is retrieved, separated by "," spos : cycle number from the beginning of the pattern execution n : number of fails"
Return Value	pin cycle@data
Return Error	None
Example	>>>GET FAILC jtag_tdo STARTC=0 NO_OF_CYCLES=100

4.13.2 GET FAILC TAG

Description	**The "GET FAILC TAG" command gets failing cycle information with start cycle and no of fails with tag. Before using this command, "EXE" command must be executed. Details see "EXE" command.**
Syntax	GET [FAILC] [FAILC_CNT] [FAILC_PIN] [TAG] [<pinlist>] [STARTC=<spos>] [NO_OF_CYCLES=n>]
Parameters	pinlist: pin list from which the result is retrieved, separated by "," spos : cycle number from the beginning of the pattern execution n : number of fails
Return Value	pin tag@cycle@data
Return Error	None
Example	>>>GET FAILC TAG STARTC=10 NO_OF_CYCLES100 >>>GET FAILC TAG pin1,pin2 STARTC=10 NO_OF_CYCLES100

4.14 Check Command

4.14.1 CHK PAT

Description	**The "CHK PAT" command checks if the pattern <label> exists.**
Syntax	`CHK PAT <label>`
Parameters	`label:` multi-port burst label or single-port label or main label
Return Value	Pattern label exist. Pattern label does not exist.
Return Error	None
Example	`>>>CHK PAT my_pattern_label` `<<<Pattern my_pattern_label does not exist.`

4.14.2 CHK MEM

Description	**The "CHK MEM" command checks the free available vector memory for a given <port>.**
Syntax	`CHK MEM <port>`
Parameters	`port:` the port name
Return Value	Number
Return Error	None
Example	`>>>CHK MEM pJTAG` `<<<111861664`

4.14.3 CHK SHELL_PROTOCOL

Description	**The "CHK SHELL_PROTOCOL" command returns the currently used SmartShell command revision.**
Syntax	`CHK SHELL_PROTOCOL`
Parameters	None
Return Value	Protocol Number
Return Error	None
Example	`>>>CHK SHELL_PROTOCOL` `<<<3`

4.14.4 CHK PAT TYPE

Description	**The "CHK PAT TYPE" command checks the pattern type.**
Syntax	`CHK PAT <label> TYPE`
Parameters	`label: any` label name
Return Value	Pattern Type burst or main , multi-port or single-port
Return Error	None
Example	`>>>CHK PAT jtag_demo TYPE` `<<<burst multi-port`

4.15 Setup Selection

4.15.1 USE OPSEQ

Description	**The "USE OPSEQ" command sets the primary <label> for the pattern execution.** **The primary label can either be a multi-port burst label or single port burst label or main label.** **It is multi-port burst label when the primary timing is multi-port timing;** **it is single-port burst label when the primary timing is a single-port multi-port timing;** **it is main label: only when the primary timing is a single-port timing.**
Syntax	`USE OPSEQ <label>`
Parameters	`label:` label can either be multi-port burst label or single port burst label or main label.
Return Value	pass no error
Return Error	ERROR: because pattern not found"
Example	`>>>USE OPSEQ my_multiport_burst_label` `<<<pass`

4.15.2 USE SPEC

Description	**The "USE SPEC" command sets a level or timing equation set or a timing specification for multi-port timing as primary setting for test execution**
Syntax	`USE SPEC [LEV]/[TIM]` `[eqnno,specnu,setno][<specification>@<setno,+>]`
Parameters	*SETS based level or timing:* `eqnno:` the number of the equation defined in level or timing equation setup `specno:` the number of the spec defined by SpecTool `setno:` the number of either as LEVELSET or TIMINGSET *Multi-port based timing:* `Specification:` defined in SpecTool `setno:` the number of TIMINGSET for each port only one setno if all ports use the same TIMINGSET #
Return Value	pass no error
Return Error	ERROR: because spec not found

Example	`USE SPEC LEV 1,1,1`
	`USE SPEC TIM 1,2,1`
	`USE SPEC TIM specs@1,1`

4.15.3 USE SETUP

Description	**The "USE SETUP" command sets a level and a timing equation set or a timing specification for multi-port timing, and burst label or pattern as primary setting for test execution**
Syntax	`USE SETUP SPEC=<levset&timset> OPSEQ=<label> TEST=<name>`
Parameters	*SETS based level or timing:*

	eqnno: the number of the equation defined in level or timing equation setup specno: the number of the spec defined by SpecTool setno: the number of either as LEVELSET or TIMINGSET *Multi-port based timing:* Specification: defined in SpecTool setno: the number of TIMINGSET for each port only one setno if all ports use the same TIMINGSET # *Specifications and Operating Sequence based setup:* SPEC: specification <level>@<timing> OPSEQ: burst label TEST: name of the test
Return Value	pass no error
Return Error	ERROR: because spec not found
Example	USE SETUP SPEC=1,1,1@1,2,1 OPSEQ=jtag_burst TEST=a_test

4.15.4 USE SUITE

Description	**"The "USE SUITE" command sets the primaries from the given test suite**
Syntax	USE SUITE <suite_name>
Parameters	suite_name: the test suite for which the primaries have to be set
Return Value	DFT_CHANGE_SUITE suite_name "DONE"
Return Error	Condition: the specified suite_name does not exist Return: ERROR: can not find test suite suite_name in current test flow!
Example	>>>USE SUITE LV <<<DFT_CHANGE_SUITE LV "DONE"

4.15.5 USE PROTOCOL

Description	**The "USE PROTOCOL" command selects the vision number of supported SmartShell command protocol**
Syntax	USE PROTOCOL <version_number>
Parameters	version_number: it can be 2 or 3
Return Value	Pass
Return Error	None
Example	>>>USE PROTOCOL 3 <<<pass

4.16 Configure Execution

4.16.1 SET SIGNAL_MAP

Description	**The "SET SIGNAL_MAP" command defines a mapping between a pin to a pin index number for "GET FAILC" and "EXE FAILC" command.**
Syntax	SET SIGNAL_MAP <pin@index>+
Parameters	pin : the pin name of the main label index: the index to that pin name
Return Value	Pass
Return Error	**Condition**: syntax error **Return**: ERROR: SYNTAX_ERROR: - command : DFT_PIN_MAP, argument : xxx
Example	>>>SET SIGNAL_MAP jtag_tdo@3 jtag_tdi@5 <<<pass

4.16.2 SET MASK

Description	**The "SET MASK" command masks the listed pins for pattern execution**
Syntax	SET MASK <pin>+
Parameters	<pin> +: list of pins separated by blank or command
Return Value	Pass
Return Error	None
Example	>>>SET MASK pin pin2 <<<pass

4.16.3 SET UNMASK

Description	**The "SET UNMASK" command unmasks the listed pins for pattern execution**
Syntax	SET UNMASK <pin>+
Parameters	<pin> +: list of pins separated by blank or comma
Return Value	pass
Return Error	None
Example	>>>SET UNMASK pin pin2 <<<pass

4.16.4 SET ERR_LIMIT

Description	**The "SET ERR_LIMIT" command sets max error count limit for failing cycles query in pattern execution**
Syntax	SET ERR_LIMIT <number>
Parameters	number : max number of errors to be captured
Return Value	Max fail error set to <number>
Return Error	None
Example	>>>SET ERR_LIMIT 1000 <<<Max fail error set to 1000

4.17 Execution Commands

4.17.1 EXE

Description	**The "EXE" command executes either main label or burst label or operating sequence**
Syntax	EXE [<[FAILC] [FAILC_CNT] [FAILC_PIN]>] [TAG] [MAIN=<main label>]/[OPSEQ=<burst label>] [STIL=<stil_file_name>][SIGNAL=<pinlist>] [STARTC=<spos>] [NO_OF_CYCLES=n>]
Parameters	FAILC : failing cycle FAILC_CNT : total fail count of all pins FAILC_PIN : failing count per pin REUSE : main label to be reused TAG : get vector comment text at failing cycle MAIN : list of main label separated by "," STIL : the converted stil file main label : list of main labels OPSEQ : burst label or operating sequence burst label : one more burst labels pinlist : pin list from which the result is retrieved, separated by "," spos : cycle number from the beginning of the pattern execution n : number of fails stil_file_name: the converted stil file's name
Return Value	Main label execution *without options:* pass or fail; *with option:* failing info. Operating sequence or burst label execution: *without options:* pass or fail; failing cycle and failing count can be retrieved by GET FAILC command; *with options:* pass or fail and failing information and failing cycle; *Format:* pass or fail ATE_FAILC cycle# pin expected result ATE_FAILC 14958 gpio_78 HHLLL LLLLL
Return Error	None
Example	>>>EXE FAILC <<<ATE_RES F <<<ATE_FAILC 14958 gpio_78 HHLLL LLLLL <<<ATE_FAILC 14959 gpio_78 HHLLL LLLLL

4.17.2 EXE FLOW

Description	**The "EXE FLOW" command executes the loaded flow**
Syntax	EXE FLOW
Parameters	None
Return Value	suite_name P/F/bypassed
Return Error	**None**
Example	<<<EXE FLOW >>>suite1 P >>>suite2 P >>>suite3 bypassed >>>suite4 F

4.17.3 EXE TO_SUITE

Description	**The "EXE TO_SUITE" command executes from the first test suite to the specified testsuite, the rest of test suites is bypassed.** **With option HOLD, SmarTest keeps the HOLD status on the specified test suite and all pins are kept connected. This is mainly used to do some other actions after the execution. You need to send command "EXE TO_SUITE CONTINUE" to continue the flow execution.** **With option CONTINUE, the flow execution is finished, the HOLD status is unset.**
Syntax	EXE TO_SUITE <suite_name>
Parameters	suite_name: an existing test suite name
Return Value	DFT_EXE_TO_SUITE demo_suite "DONE" The executed test suites and the results
Return Error	**Condition:** the specified suite_name does not exist **Return:** ERROR: SYNTAX_ERROR: - command : DFT_EXEC_TO_SUITE test_suite_name
Example	<<<EXE TO_SUITE LV >>>DFT_EXE_TO_SUITE demo_suite "DONE" >>>suite1 P >>>suite2 F

4.17.4 EXE SUITE

Description	**The "EXE SUITE" command executes an existing test suite**
Syntax	EXE SUITE <suite>
Parameters	suite: an existing test suite name
Return Value	ATE_RES P or ATE_RES F
Return Error	Condition: if test suite name does not exist Return:
Example	<<<EXE SUITE suite_demoo >>>ATE_RES P

4.18 EDF Control

4.18.1 SET EDF_ON

Description	**The "SET EDF_ON" command starts to record the edf log**
Syntax	SET EDF_ON
Parameters	None
Return Value	Pass
Return Error	None
Example	>>>SET EDF_ON <<<pass

4.18.2 SET EDF_OFF

Description	**The "SET EDF_OFF" command stops to record the edf log**
Syntax	SET EDF_OFF
Parameters	None
Return Value	pass
Return Error	None
Example	>>>SET EDF_OFF <<<pass

4.18.3 GET EDF

Description	**The "GET EDF" command gets the recorded edf info**
Syntax	GET EDF
Parameters	None
Return Value	the recorded edf info
Return Error	**Condition**: you use this command before do not use command "SET EDF_ON" **Return**: WARNING: no results is found
Example	>>>GET EDF <<<DFT_QUERY_ATE_INFO EDF ConfigurationChangedEventType with 5 attributes received => received =>Attribute type: 0 …… 44: TCK ( ) 45: TRSTN ( ) 3: all_inputs (34 pins): 0: BADD0 ( )

4.19 Site Control

4.19.1 GET SITE_FOCUS

Description	**The "GET SITE_FOCUS" command returns the focused site number**
Syntax	GET SITE_FOCUS
Parameters	None
Return Value	the focus site
Return Error	None
Example	>>>GET SITE_FOCUS <<<2

4.19.2 SET SITE_FOCUS

Description	**The "SET SITE_FOCUS" command sets the focused site (the site should have been enabled)**
Syntax	SET SITE_FOCUS <site_number>
Parameters	None
Return Value	pass no error
Return Error	Error message the given <site_number> is out of range
Example	>>>SET SITE_FOCUS 2 <<<pass

4.19.3 SET SITE_ENABLE ALL

Description	**The "SET SITE_ENABLE ALL" command enables all the sites**
Syntax	SET SITE_ENABLE ALL
Parameters	None
Return Value	pass no error
Return Error	None
Example	>>>SET SITE_ENABLE ALL <<<pass

4.19.4 SET SITE_ENABLE

Description	**The "SET SITE_ENABLE" command enables the specified site on focus**
Syntax	SET SITE_ENABLE <ALL|site_id>
Parameters	"ALL" or site_id
Return Value	pass no error
Return Error	None
Example	>>>SET SITE_ENABLE <<<pass

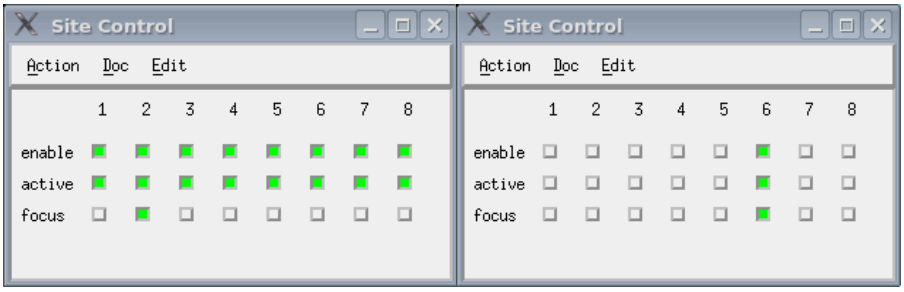


Fig. 7: Site control pop-up window before and after the command

4.19.5 SET SITE_DISABLE

Description	**The "SET SITE_DISABLE" command disables all the sites or the specified site**
Syntax	SET SITE_DISABLE <ALL|site_id>
Parameters	"ALL" or site_id
Return Value	pass no error
Return Error	None
Example	>>>SET SITE_DISABLE ALL <<<pass

4.20 Suite Control

4.20.1 GET SUITE SETUP

Description	**The "GET SUITE SETUP" command returns a list of the test setup information for all test suites in the loaded testflow or the test setup information of a specified test suite of the loaded testflow**
Syntax	GET SUITE SETUP [<suite_name>]
Parameters	suite_name: the test suite name defined in the test flow
Return Value	One or a list of test suite setup info
Return Error	**Condition**: the given test suite name does not exist **Return**: ERROR: test suite "<suite_name>" does not exist
Example	>>>GET SUITE SETUP jtag_demo <<<timing info <<<spec: "spec_pJTAG_STD_spec" <<<timset: "1" <<<level info <<<eqn: 1 <<<spec: 1 <<<levelset: 1 <<<pattern info <<<pattern: jtag_demo

4.20.2 GET SUITE SETUP PARAM

Description	**The "GET SUITE SETUP PARAM" command returns the test setup and TestMethod information of a specified test suite of the loaded testflow**
Syntax	GET SUITE SETUP <suite_name> PARAM
Parameters	suite_name: the test suite name defined in the test flow
Return Value	Suite setup and TestMethod info
Return Error	**Condition**: the test suite does not exist **Return**: ERROR: test suite "<suite_name>" does not exist
Example	>>> GET SUITE SETUP demo_suite PARAM <<<suite demo_suite: <<< timing info <<< eqn: 1 <<< spec: 1 <<< timset: 1 <<< level info <<< eqn: 1 <<< spec: 1 <<< levset: 1

	<<< pattern info <<< pattern: demo1 <<< TestMethod info <<< TestMethodname:ac_tml.AcTest.FunctionalTest <<< testName:Functional <<< output:None

4.20.3 GET SETUP_FILE

Description	**The "GET SETUP_FILE" command gives the loaded setup files**
Syntax	GET SETUP_FILE
Parameters	None
Return Value	The loaded setup files
Return Error	None
Example	>>>GET SETUP_FILE <<<DFT_QUERY_ATE_INFO SETUP_FILE <<<testflow : smartshell_demo.flw <<<pin : smartshell_demo.pin <<<level : smartshell_demo.lvl <<<timing : smartshell_demo.tim.mfh <<<pattern : smartshell_demo.pmfl

4.20.4 DELETE SUITE

Description	**The "DELETE SUITE" command deletes the given test suite**
Syntax	DELETE SUITE <suite_name>
Parameters	suite_name: existing test suite name
Return Value	None
Return Error	None
Example	>>>DELETE SUITE jtag_demo <<<pass

4.21 COMMAND RECORD

4.21.1 LOG CMD

Description	**The "LOG CMD" command defines the prefix string of unrecord commands, then the relevant commands are recorded in the log files.**
Syntax	LOG CMD [ALL] [<cmd_prefix_name>]
Parameters	The first word of command, like “GET”, “SET” and so on
Return Value	None
Return Error	None
Example	>>>LOG CMD GET <<<pass

4.21.2 LOG STR

Description	**The "LOG STR" command records the string in the log file but do nothing**
Syntax	LOG STR <string>
Parameters	string: any string
Return Value	Pass
Return Error	None
Example	>>>LOG STR start to debug <<<pass

4.21.3 UNLOG CMD

Description	**The "UNLOG CMD" command defines the prefix string of unrecord commands, then the relevant commands are not recorded in the log files.**
Syntax	UNLOG CMD [ALL] [<cmd_prefix_name>]
Parameters	The specified command.
Return Value	Pass
Return Error	None
Example	>>>UNLOG CMD GET <<<pass